소프트웨어 디자인 명세서

ASKku - 개인 맞춤형 학교 정보 검색 플랫폼

소프트웨어공학개론 1 조

김태완, 강철석, 김건, 오은서, 이동환, 이석원

Contents

1. Purpose 7

1.1. Readership 7

1.2. Scope 7

1.3. Objectives 7

1.4. Document Structure 7

2. Introduction 9

2.1. Objectives 9

2.2 Applied Diagrams 9

2.2.1. Used Tools 9

2.2.2. Use Case Diagram 9

2.2.3. Sequence Diagram 9

2.2.4. Class Diagram 10

2.2.5. Context Diagram 10

2.2.6. Entity Relationship Diagram 10

2.2.7. Project Scope 10

2.2.8. References 10

3. System Architecture – Overall 11

3.1. Objectives 11

3.2. System Organization 11

3.2.1. System Diagram 12

3.3. Use Case Diagram 13

4. System Architecture – Frontend 14

4.1 Objectives 14

4.1.1. User Profile Component 14

4.1.2. Conversation & Message Component 15

4.1.3. Timetable Component 16

4.1.4. RAG Notice Retrieval Component 18

4.1.5. Bookmark Component 19

4.1.6. Schedule Component 20

5. System Architecture – Backend 22

5.1. Objectives 22

5.2. Overall Structure 22

5.3. Subcomponents 22

5.3.1. User Management System 22

5.3.2. Query Processing System (Conversation & Message System) 23

5.3.3. Notice Retrieval System (RAG Data Layer) 24

5.3.4. Bookmark & Schedule System 25

6. Protocol Design 27

6.1. Objectives 27

6.2. AJAX 27

6.3. Transport Layer 27

6.4. Authentication 27

6.4.1. Register 27
6.4.2. Login 28
6.4.3. Email Duplication Check 28
6.5. Timetable API 29
6.5.1. Create Timetable 29
6.5.2. Add Timetable Item 29
6.5.3. Delete Timetable Item 30
6.6. Notice & RAG Retrieval API 30
6.7. Conversation + RAG API 31
6.8. Bookmark Management API 31
6.8.1. Add Bookmark 31
6.9. Schedule Update System API 32
7. Database Design 33
7.1. Objectives 33
7.2. ER Diagram 33
7.2.1. User 34
7.2.2. Conversation 34
7.2.3. Message 35
7.2.4. Bookmark 35
7.2.5. Schedule 36
7.2.6. Timetable 36
7.2.7. Timetable_item 37
7.3. Relational Schema 37
7.4. SQL DDL 38
7.4.1. User 38
7.4.2. Conversation 38
7.4.3. Message 39
7.4.4. Bookmark 39
7.4.5. Schedule 40
7.4.6. Timetable 40
7.4.7. Timetable_item 41
8. Testing Plan 42
8.1. Objectives 42
8.2. Testing Policy 42
8.2.1. Development Testing 42
8.2.2. Release Testing 43
8.2.3. User Testing 43
8.2.4. Testing Case 44
9. Development Plan 45
9.1. Objectives 45
9.2. Frontend Environment 45
9.2.1. React 45
9.2.2. HyperText Markup Language (HTML) 45

9.2.3. Cascading Style Sheets (CSS) 45
9.3. Backend Environment 45
9.3.1. Node.js API Server 46
9.3.2. Python RAG Module 46
9.3.3 Database (MySQL & ChromaDB) 46
9.4 Constraints 46
9.5. Assumptions and Dependencies 47
10. Supporting Information 48
10.1. Software Design Specification 48
10.2. Document History 48

List of Figures

Figure 3.1: Overall System Architecture 11
Figure 3.2: Sequence Diagram – Overall 12
Figure 3.3: Use Case Diagram – Overall 13

Figure 4.1: Class Diagram – User Profile 14
Figure 4.2: Sequence Diagram - User Profile 15
Figure 4.3: Class Diagram - Conversation 16
Figure 4.4: Sequence Diagram – Conversation 16
Figure 4.5: Class Diagram - Timetable 17
Figure 4.6: Sequence Diagram – Timetable 17
Figure 4.7: Class Diagram - RAG 18
Figure 4.8: Sequence Diagram - RAG 18
Figure 4.9: Class Diagram – Bookmark 19
Figure 4.10: Sequence Diagram – Bookmark 20
Figure 4.11: Class Diagram - Schedule 21
Figure 4.12: Sequence Diagram - Schedule 21

Figure 5.1: Overall Structure 22
Figure 5.2: Class Diagram – User Management 23
Figure 5.3: Sequence Diagram – User Management 23
Figure 5.4: Class Diagram – Query Processing 24
Figure 5.5: Sequence Diagram – Query Processing 24
Figure 5.6: Class Diagram – Notice 25
Figure 5.7: Sequence Diagram – Notice 25
Figure 5.8: Class Diagram – Bookmark & Schedule 26
Figure 5.9: Sequence Diagram – Bookmark & Schedule 26

Figure 6.1: Table of Transport Layer 27
Figure 6.2: Table of Register Request 28
Figure 6.3: Table of Register Response 28
Figure 6.4: Table of Login Request 28
Figure 6.5: Table of Login Response 28
Figure 6.6: Table of Email Duplication Request 28
Figure 6.7: Table of Email Duplication Response 29
Figure 6.8: Table of Timetable Create Request 29
Figure 6.9: Table of Timetable Create Response 29
Figure 6.10: Table of Add Timetable Item Request 29
Figure 6.11: Table of Add Timetable Item Response 30
Figure 6.12: Table of Delete Timetable Item Request 30
Figure 6.13: Table of Delete Timetable Item Response 30

Figure 6.14: Table of Notice & RAG Retrieval Request 30
Figure 6.15: Table of Notice & RAG Retrieval Response 30
Figure 6.16: Table of Conversation & RAG Request 31
Figure 6.17: Table of Conversation & RAG Response 31
Figure 6.18: Table of Add Bookmark Request 31
Figure 6.19: Table of Add Bookmark Response 31
Figure 6.20: Table of Schedule Update Request 32
Figure 6.21: Table of Schedule Update Response 32

Figure 7.1: ER Diagram 33
Figure 7.2: ER Diagram, Entity, User 34
Figure 7.3: ER Diagram, Entity, Conversation 34
Figure 7.4: ER Diagram, Entity, Message 35
Figure 7.5: ER Diagram, Entity, Bookmark 35
Figure 7.6: ER Diagram, Entity, Schedule 36
Figure 7.7: ER Diagram, Entity, Timetable 36
Figure 7.8: ER Diagram, Entity, Timetable_item 37
Figure 7.9: Relational Schema 37
Figure 7.10: User SQL schema 38
Figure 7.11: Conversation SQL schema 38
Figure 7.12: Message SQL schema 39
Figure 7.13: Bookmark SQL schema 39
Figure 7.14: Schedule SQL schema 40
Figure 7.15: Timetable SQL schema 40
Figure 7.16: Timetable_item SQL schema 41

1. Purpose

본 문서는 성균관대학교 구성원을 위한 정보 검색 플랫폼 'ASKku'의 소프트웨어 설계 명세서이다. 본 장에서는 문서의 예상 독자와 시스템의 범위, 설계 목적, 그리고 전체적인 문서 구조에 대해 설명한다.

1.1. Readership

본 문서는 다음과 같은 독자를 대상으로 한다. 우선 본 시스템의 개발자들(소프트웨어공학개론 1조)로, 프론트엔드 개발자와 백엔드 개발자, 그리고 크롤러·RAG 파이프라인 등 AI 관련 모듈을 구현하는 개발자로 구성된다. 또한 본 과목을 담당하는 교수와 조교, 그리고 수업에 참여하는 학생을 주요 독자로 한다.

1.2. Scope

프론트엔드·백엔드 아키텍처, 클라이언트–서버 통신 프로토콜, 데이터베이스 구조, 테스트 및 개발 계획 등 시스템 구현에 필요한 설계 사항을 정리하며, 세부 구현 코드나 사용자 매뉴얼은 포함하지 않는다.

1.3. Objectives

본 문서의 목적은 ASKku 시스템의 기술적 설계를 제시하고, 프론트엔드·백엔드 아키텍처, 클라이언트–서버 통신, 데이터베이스 측면 등 구현을 위한 기준을 제공하는 것이다. 또한 시스템 구조, 주요 데이터 흐름 및 구성 요소 간 인터페이스를 정리함으로써 개발 단계에서의 혼선을 줄이는 것을 목표로 한다.

1.4. Document Structure

1) Purpose: 본 문서의 독자층, 범위, 목적 및 전체 구조를 소개한다.
2) Introduction: 본 문서 작성에 사용된 도구와 다이어그램, 참고 자료들에 대해 설명한다.
3) Overall System Architecture: 시스템의 전체적인 구조를 Context Diagram, Use-case Diagram, Sequence Diagram을 이용하여 제시한다.
4) System Architecture – Frontend: 프론트엔드 시스템의 구조를 Class Diagram, Sequence Diagram을 이용하여 설명한다.
5) System Architecture – Backend: 백엔드 시스템의 구조를 Sequence Diagram, Class Diagram을 이용하여 설명한다.

6) Protocol Design: 프론트엔드 애플리케이션과 서버 간의 요청/응답 형식 및 통신 프로토콜을 서술한다.

7) Database Design: ER Diagram과 Relation Schema를 이용하여 시스템의 데이터베이스 디자인을 설명한다.

8) Testing Plan: 시스템을 위한 테스트 계획을 서술한다.

9) Development Plan: 시스템의 구현 계획 및 구현을 위한 개발 도구, 라이브러리, 외부 의존성 등의 개발 환경을 설명한다.

10) Supporting Information: 본 문서의 작성 및 수정 기록을 기술한다.

2. Introduction

본 설계 문서는 ASKku 시스템 구현의 기반이 되는 상세 설계를 제공하며, 앞서 작성된 소프트웨어 요구사항 명세서(SRS)에 명시된 요구사항을 바탕으로 작성되었다.

2.1. Objectives

본 장에서는 ASKku 시스템 설계에 사용된 다이어그램과 작성 도구를 소개하고, 각 다이어그램 유형의 역할을 간략히 설명한다. 아울러 프로젝트의 범위를 정리하고, 설계 과정에서 참조한 문서와 자료를 명시함으로써 이후 장에서 제시되는 아키텍처와 상세 설계를 이해하기 위한 기초를 제공한다.

2.2 Applied Diagrams

2.2.1. Used Tools

본 설계 문서의 다이어그램 작성에는 PlantUML, draw.io, mermaid를 활용하였다. PlantUML과 mermaid는 텍스트 기반으로 다이어그램을 정의할 수 있어 설계 변경 사항을 버전 관리 시스템과 함께 추적하기에 용이하며, draw.io는 브라우저에서 직관적으로 편집 가능한 그래픽 다이어그램 도구로 팀원 간 협업과 수정에 적합하다. 이러한 도구들을 통해 프로젝트 전반의 다이어그램을 일관된 형식으로 관리하고, 변경된 설계 내용을 문서에 신속히 반영할 수 있도록 하였다.

2.2.2. Use Case Diagram

Use Case 다이어그램은 시스템이 제공해야 하는 기능과 서비스를 사용자 관점에서 정리한 다이어그램으로, 액터와 각 유스케이스 간의 관계를 통해 사용자–시스템 간 상호작용을 표현한다. ASKku의 Use Case 다이어그램에서는 추상적인 User 아래에 Unregistered User와 Registered User를 구분하고, 회원가입 이전에는 공지 열람과 회원가입만, 로그인 이후에는 계정 관리, 공지·북마크 관리, 개인화 Q&A, 일정·시간표 관리 등 주요 기능을 사용할 수 있도록 모델링하였다. 또한 개인화 Q&A 유스케이스는 벡터/검색 스토어와 LLM API와의 상호작용을 포함하는 RAG 기반 정보 검색 과정을 나타내어, 시스템이 외부 검색·언어 모델 서비스를 활용해 답변을 생성함을 보여준다.

2.2.3. Sequence Diagram

시퀀스 다이어그램은 시간 순서에 따라 객체들 사이에서 어떤 상호작용이 일어나는지를 나타내는 다이어그램이다. Event Diagram이라고도 하며, 특정 시나리오를 수행하기 위해 관련된 객체들이 주고받는 메시지를 순서대로 표현한다. 본 설계 문서에서는 주요 시나리오에 대해 시퀀스 다이어그램을 제시하여, RAG 기반 질의응답 흐름을 포함한 클라이언트–서버 간 구체적인 상호작용과 각 구성 요소의 역할을 설명한다.

2.2.4. Class Diagram

클래스 다이어그램은 시스템을 구성하는 클래스와 그 속성, 기능, 그리고 클래스 간의 관계를 표현하여 시스템의 정적인 구조를 보여주는 다이어그램이다. 이를 통해 시스템이 어떤 객체들로 구성되어 있고, 각 객체에 어떤 책임이 배분되어 있으며, 서로 어떻게 연관·상속·의존 관계를 맺는지 한눈에 파악할 수 있다. 본 설계 문서에서는 ASKku 시스템의 주요 도메인 객체와 모듈 간의 구조를 클래스 다이어그램으로 제시한다. 이 다이어그램은 실제 구현되는 소스 코드 구조와 다를 수 있으며, 의미나 해석 또한 경우에 따라 달라질 수 있다.

2.2.5. Context Diagram

컨텍스트 다이어그램은 데이터 흐름 다이어그램(DFD) 중 가장 상위 수준의 다이어그램으로, 시스템 전체와 그 시스템과 상호작용하는 외부 엔티티, 그리고 그 사이를 오가는 주요 데이터 흐름을 한눈에 보여준다. ASKku의 컨텍스트 다이어그램에서는 ASKku를 하나의 프로세스로 두고, 사용자, 성균관대학교의 웹 정보원, 외부 LLM API 등과의 입·출력 관계를 표현하여 시스템 경계와 통합 지점을 나타낸다. 이 다이어그램은 세부 구현보다는 시스템의 윤곽과 외부 연계를 설명하는 데 초점을 두며, 개발자뿐 아니라 비기술 이해관계자도 전체 구조를 이해하는 데 활용할 수 있다.

2.2.6. Entity Relationship Diagram

ER 다이어그램은 구조화된 데이터와 그들 간의 관계를 사람이 이해하기 쉬운 형태로 표현하는 다이어그램이다. 데이터베이스 설계 과정에서 개념적 데이터 모델을 나타내며, 엔티티(Entity), 속성(Attribute), 관계(Relationship)으로 구성된다. 이 다이어그램을 통해 ASKku 시스템에서 사용하는 사용자, 문서, 질의·응답, 일정 등의 데이터가 어떻게 저장되고 서로 어떤 관계를 맺는지 논리적 구조를 설명한다.

2.2.7. Project Scope

ASKku 프로젝트는 성균관대학교의 여러 웹사이트에 흩어져 있는 학교 정보를 통합하고, 개별 사용자의 학번·학과·캠퍼스 정보에 기반한 맞춤형 검색과 질의응답을 제공하는 웹 기반 플랫폼이다. 사용자는 자연어로 질문을 입력해 관련 공지 및 학교 정보를 한 번에 조회할 수 있으며, 개인 시간표와 일정을 관리하면서 자신에게 필요한 정보를 더 효율적으로 탐색할 수 있다. 본 프로젝트는 정보 검색·대화형 Q&A·일정 관리 기능에 초점을 두며, 수강신청이나 성적 조회와 같은 학사 행정 처리나 사용자 간 커뮤니티 기능은 범위에서 제외한다.

2.2.8. References

- IEEE Std 1016-2009, IEEE Recommended Practice for Software Design Descriptions.
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- ASKku 소프트웨어 요구사항 명세서, 소프트웨어공학개론 1조, 2025.
- OpenAI, ChatGPT API Documentation, 2025.
- LangChain, LangChain Documentation, 2025.

3. System Architecture – Overall

3.1. Objectives

본 장의 목적은 ASKku 시스템의 전체적인 아키텍처를 개괄적으로 설명하는 것이다. System Organization과 System Diagram을 통해 주요 구성 요소와 외부 시스템 간의 관계를 제시하고, Use Case Diagram을 통해 사용자 관점에서 시스템이 제공하는 주요 기능과 상호작용을 정리한다. 이를 통해 이후 프론트엔드·백엔드 상세 설계를 이해하기 위한 상위 수준의 공통 아키텍처 관점을 제공한다.

3.2. System Organization

ASKku 시스템은 웹 기반 클라이언트–서버 구조를 따르며, 크게 프론트엔드, 백엔드, 데이터/외부 서비스 계층의 세 부분으로 구성된다. 프론트엔드는 웹 브라우저에서 동작하는 사용자 인터페이스로, 사용자가 공지 조회, 질의응답(Chat), 마이페이지(시간표·일정 관리) 등의 기능을 이용할 수 있는 화면을 제공한다.

백엔드는 프론트엔드로부터 전달된 로그인, 질문, 일정 관리 등의 요청을 처리하는 애플리케이션 서버로, REST API를 통해 클라이언트와 JSON 형식으로 데이터를 주고받는다. 질문 처리 기능은 RAG 기반 파이프라인을 통해 관련 문서를 검색하고 외부 LLM API를 호출하여 최종 답변을 생성한 뒤, 그 결과를 프론트엔드에 반환한다.

데이터/외부 서비스 계층에는 사용자·일정·대화 이력 등을 저장하는 관계형 데이터베이스와, 문서 임베딩을 저장하는 벡터 스토어, 성균관대학교의 여러 웹 정보원 및 LLM API와 같은 외부 서비스가 포함된다. 백엔드는 이 계층과 상호작용하여 최신 학교 정보를 수집·갱신하고, 개인화된 검색과 질의응답에 필요한 데이터를 조회·저장함으로써 ASKku의 주요 기능을 지원한다.

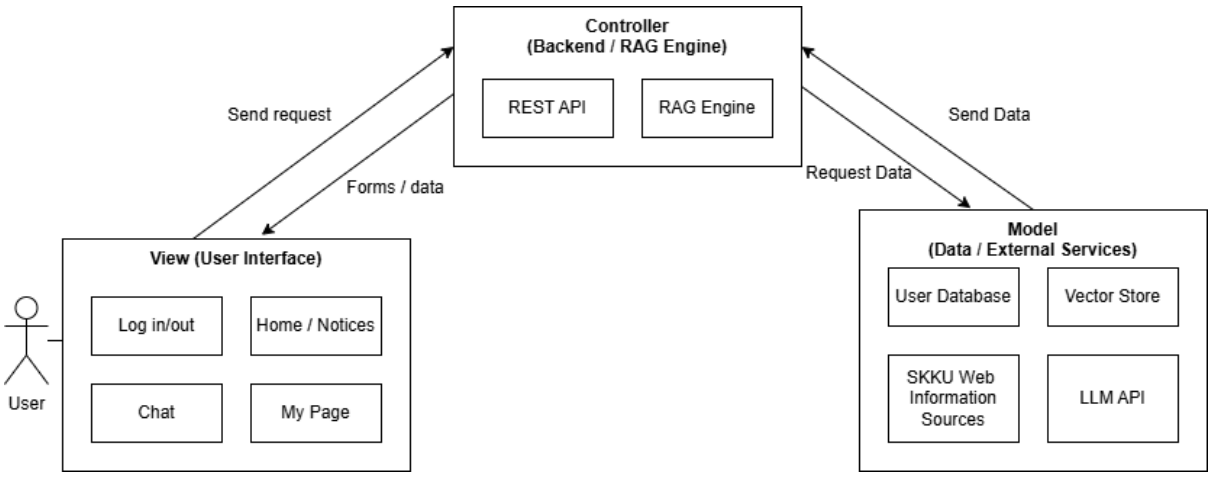

Figure 3.1: Overall System Architecture

3.2.1. System Diagram

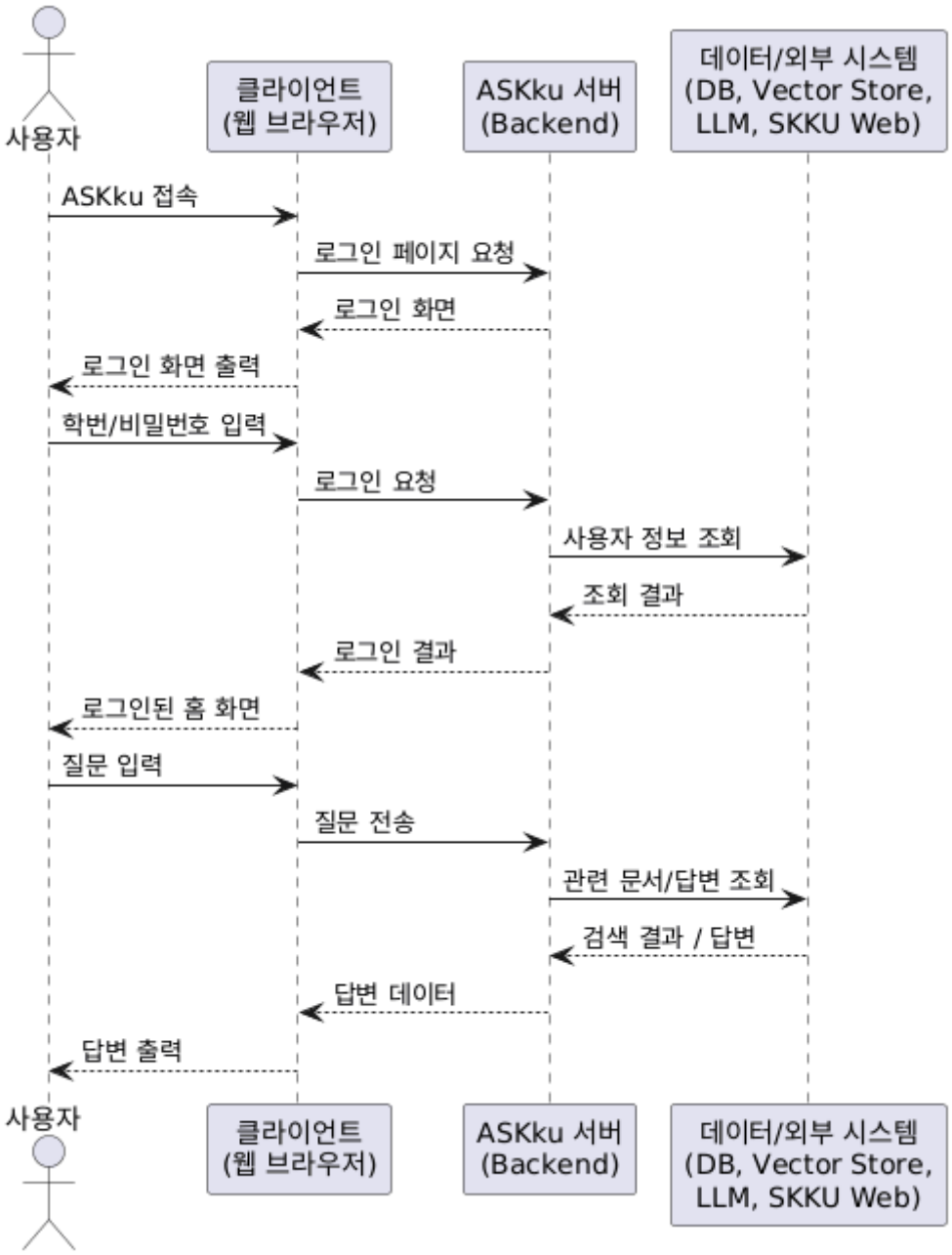

Figure 3.2: Sequence Diagram – Overall

3.3. Use Case Diagram

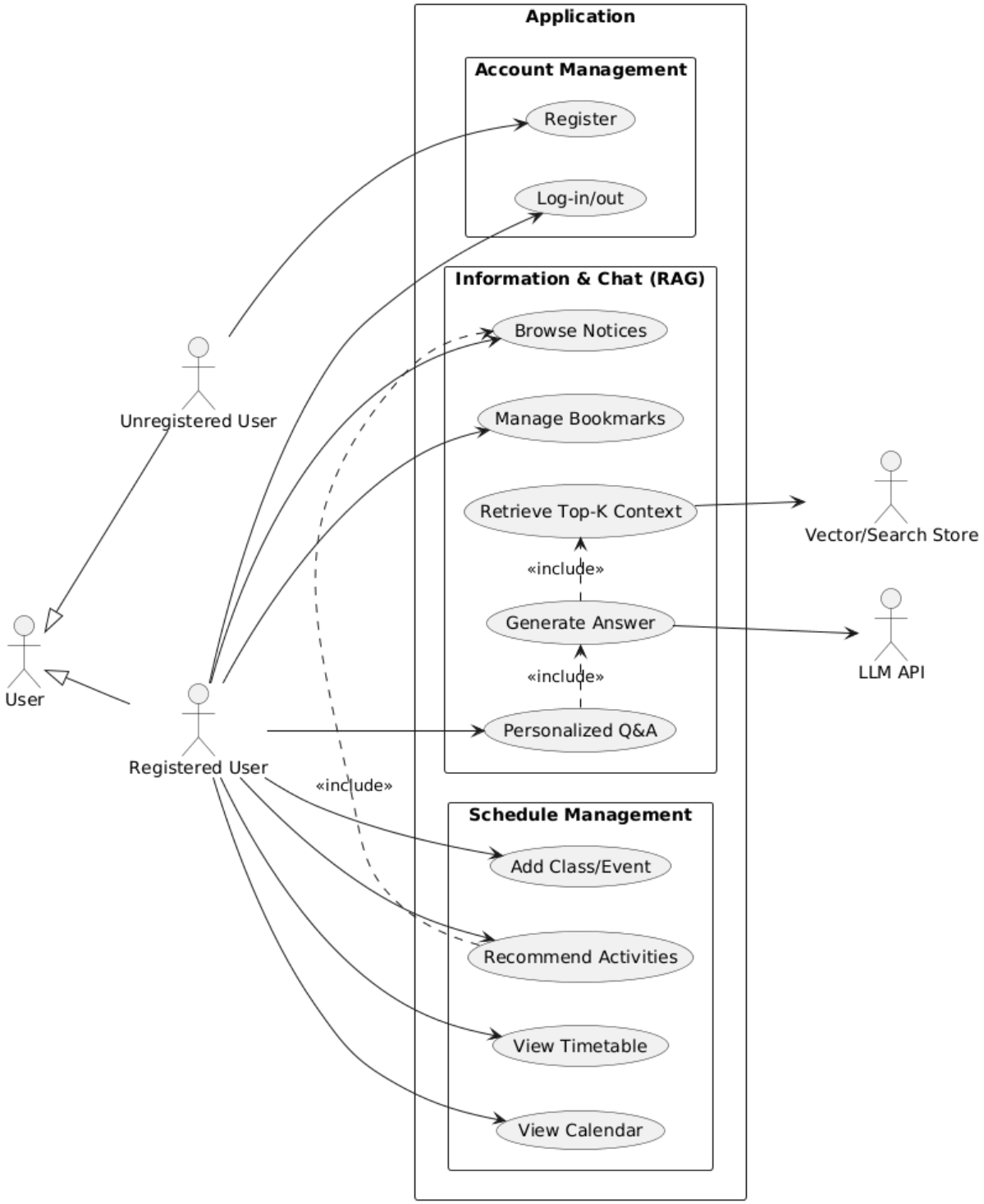

Figure 3.3: Use Case Diagram – Overall

4. System Architecture – Frontend

4.1 Objectives

이 장에서는 프론트엔드 시스템의 구조, 속성 및 기능을 설명하고 ASKku 서비스에서 각 구성 요소 간 관계를 정의한다. 특히 사용자 프로필 관리, 대화 기반 질의 처리, 사용자 상태 저장 및 인터페이스 간 데이터 흐름을 중심으로 프론트엔드 동작 방식을 다룬다.

4.1.1. User Profile Component

User Profile 컴포넌트는 사용자 계정 정보를 관리하며, 인증 및 개인화 기능의 핵심 역할을 수행한다. 사용자가 회원가입을 완료하면 해당 정보는 백엔드 MySQL DB에 저장되며, 로그인 후 JWT 기반 인증을 통해 보호된 API 접근이 가능해진다. 가입 이후 사용자는 학번(또는 계정 식별자)을 제외한 대부분의 정보를 수정할 수 있다.

Objectives

- userID (Primary Key): 사용자 식별자
- email: 사용자의 이메일
- name: 사용자 이름
- department: 소속 학과
- password_hash: 사용자 인증용 암호화된 비밀번호

Methods

- createProfile()
- updateProfile()
- getProfile()

Class Diagram

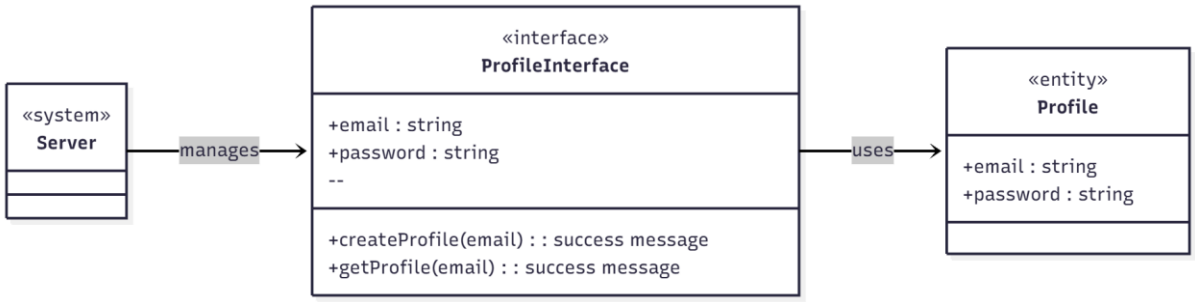

Figure 4.1: Class Diagram – User Profile

Sequence Diagram

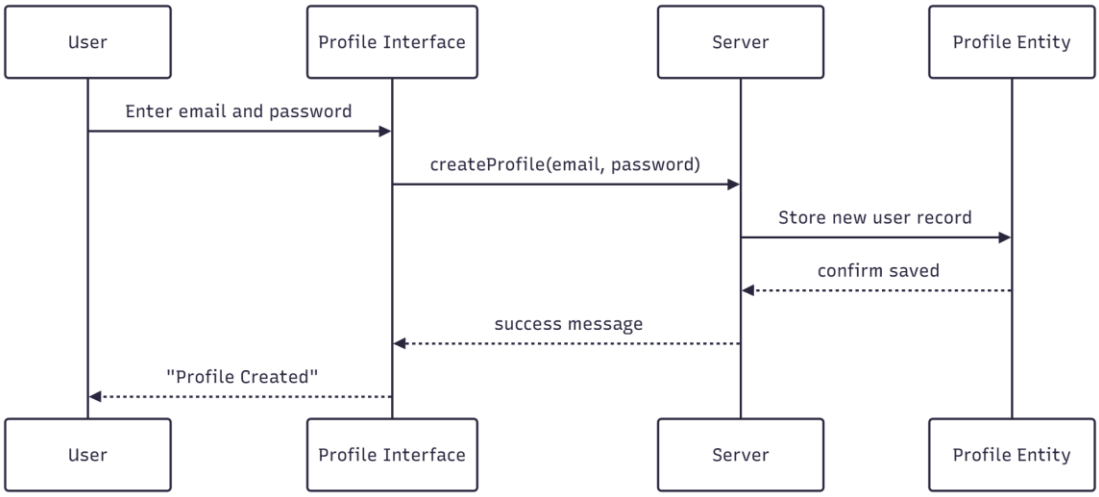

Figure 4.2: Sequence Diagram - User Profile

4.1.2. Conversation & Message Component

Conversation 시스템은 사용자의 AI 챗봇 대화 흐름을 저장하고 관리한다. 하나의 Conversation은 여러 Message 객체로 구성되며, 메시지는 사용자 입력과 AI(assistant)의 응답을 포함한다. RAG 서버와 통신하여 데이터 기반 응답을 생성하고 저장하는 것이 핵심 기능이다.

Conversation Objectives

- convID (Primary Key): 개별 대화 세션 구분용 고유 식별자
- userID: 사용자 식별자
- title: 대화 제목 또는 첫 메시지 기반 자동 생성 라벨
- createdAt: 생성 시각
- updatedAt: 마지막 메시지 저장 시각

Message Objectives

- messageID (Primary Key): 메시지를 식별하는 키
- convID (Foreign Key): 메시지가 포함된 대화 세션
- userID (nullable for assistant messages): 메시지 작성 사용자
- role (user or assistant): 메시지 발신 주체
- message: 실제 텍스트 기반 대화 내용
- timestamp: 메시지 생성 시각

Methods

- createConversation()
- sendMessage()
- appendResponse()
- loadConversationHistory()

Class Diagram

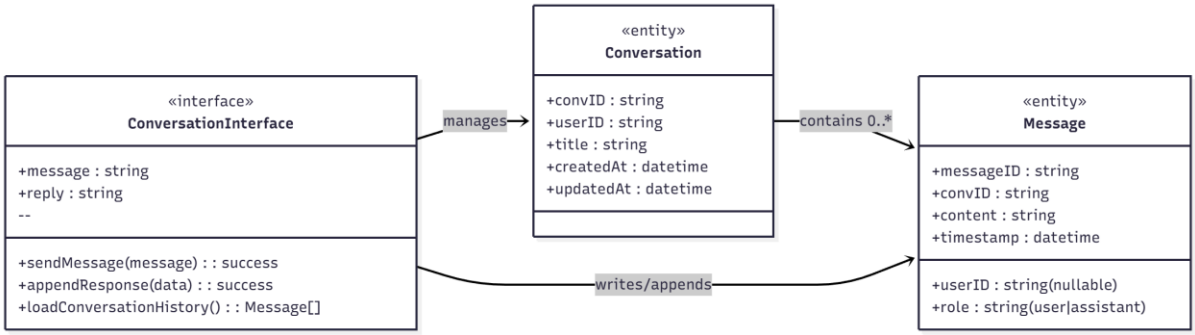

Figure 4.3: Class Diagram - Conversation

Sequence Diagram

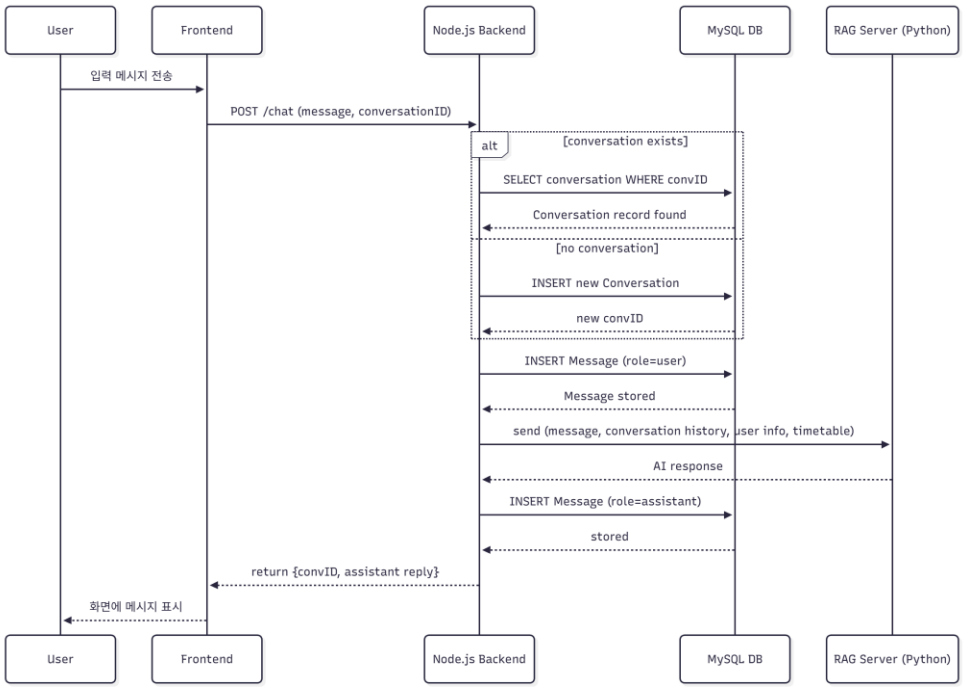

Figure 4.4: Sequence Diagram – Conversation

4.1.3. Timetable Component

Timetable Component는 사용자가 저장한 강의 시간표 데이터를 관리한다. 시간표 정보는 백엔드 데이터베이스에 저장되며, 사용자의 질문이 AI로 전달될 때 개인화 응답 생성 컨텍스트로 포함된다.

Timetable Objectives

- timetableID (Primary Key): 개별 시간표 구분 식별자
- userID (Foreign Key): 사용자 식별자
- season: 시간표가 속한 학기

TimetableItem objectives

- itemID (Primary Key): 시간표 항목 구분 식별자
- timetableID (Foreign Key): 해당 항목이 속한 시간표
- courseName: 강의명
- dayOfWeek: 수업 요일
- startTime / endTime: 강의 시작/종료 시간
- location: 강의실 위치

Methods

- createTimetable()
- updateTimetable()
- getTimetable()
- deleteTimetable()

Class Diagram

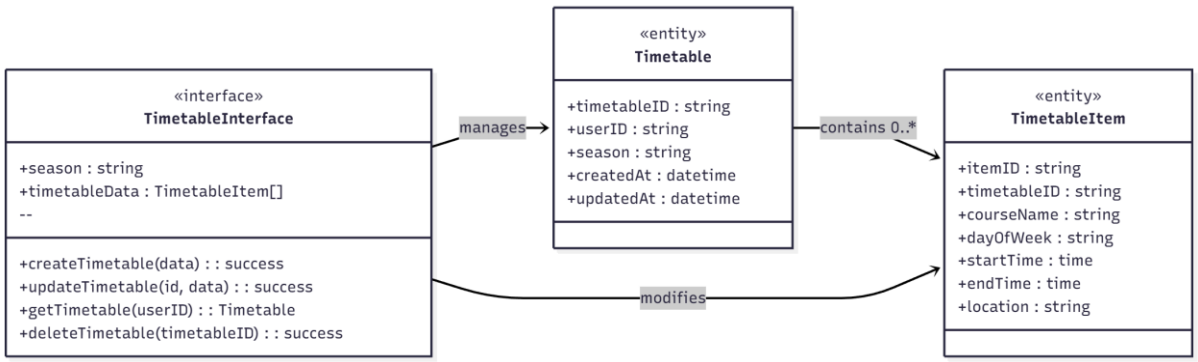

Figure 4.5: Class Diagram - Timetable

Sequence Diagram

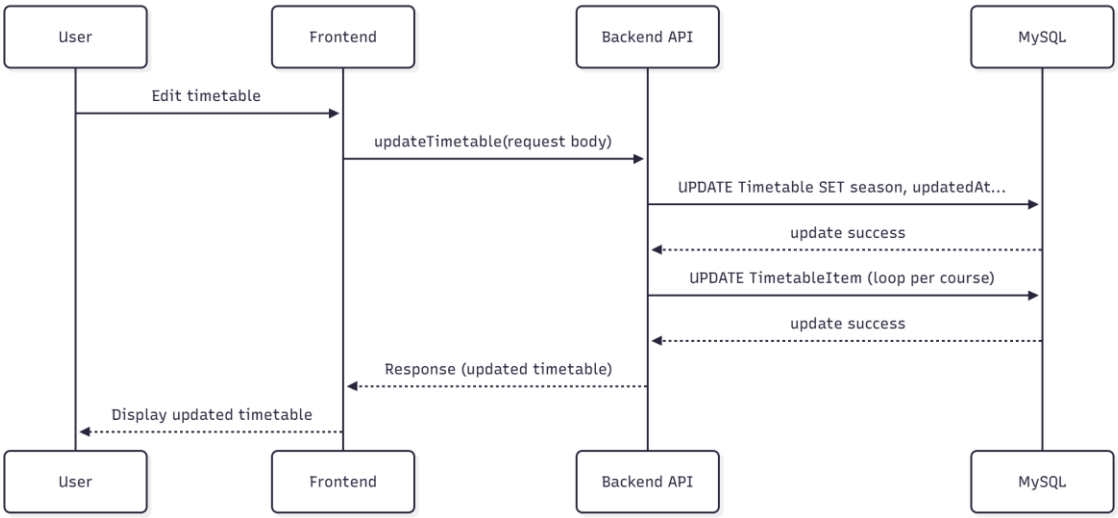

Figure 4.6: Sequence Diagram – Timetable

4.1.4. RAG Notice Retrieval Component

Notice Retrieval Component는 학과 공지사항, 학사 자료, PDF 학습 자료 등 외부 데이터를 처리 및 검색하는 AI 기능을 담당한다. Python 기반 RAG 서버가 크롤링된 데이터와 벡터 DB(ChromaDB)에 저장된 문서를 검색한 뒤, LLM을 활용하여 자연어 응답을 생성한다.

Objectives

- retriever: 벡터 검색 엔진
- embeddingModel: 공지 제목 및 본문
- ragServerURL: 공지 등록 혹은 크롤링된 시간

Methods

- search(query)
- retrieveContext()
- generateLLMResponse()
- refreshDataSources()

Class Diagram

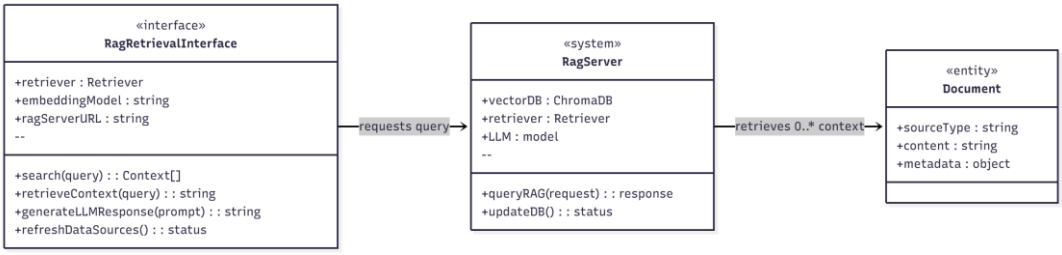

Figure 4.7: Class Diagram - RAG

Sequence Diagram

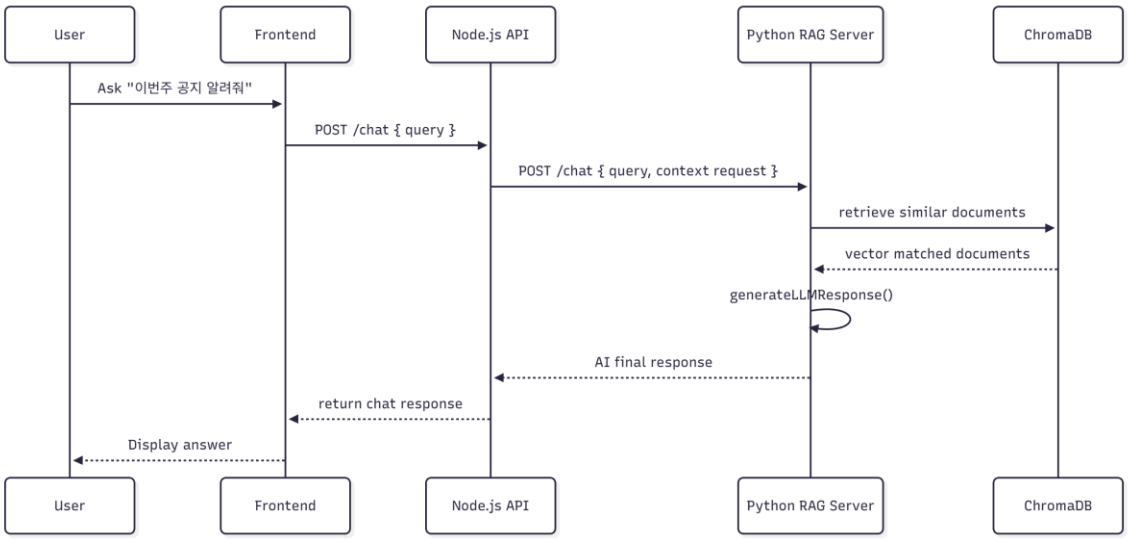

Figure 4.8: Sequence Diagram - RAG

4.1.5. Bookmark Component

Bookmark 컴포넌트는 사용자가 대화 내용 중 중요한 메시지를 저장하는 기능을 제공한다. 북마크된 메시지는 이후 검색, 다시보기, 요약 기반 스케줄 생성 등 사용자 편의 기능과 연계된다. 북마크 정보는 백엔드 DB에 저장되며, 프론트엔드는 해당 데이터를 기반으로 정렬, 필터링 및 하이라이팅 UI를 구성한다.

Objectives

- markID (Primary Key): 북마크 식별자
- userID: 북마크 소유 사용자
- title: 사용자가 지정하거나 자동 생성된 제목
- contents: 저장된 메시지 또는 LLM 요약 결과
- createdAt

Methods

- addBookmark()
- getBookmark()
- updateBookmark()
- deleteBookmark()

Class Diagram

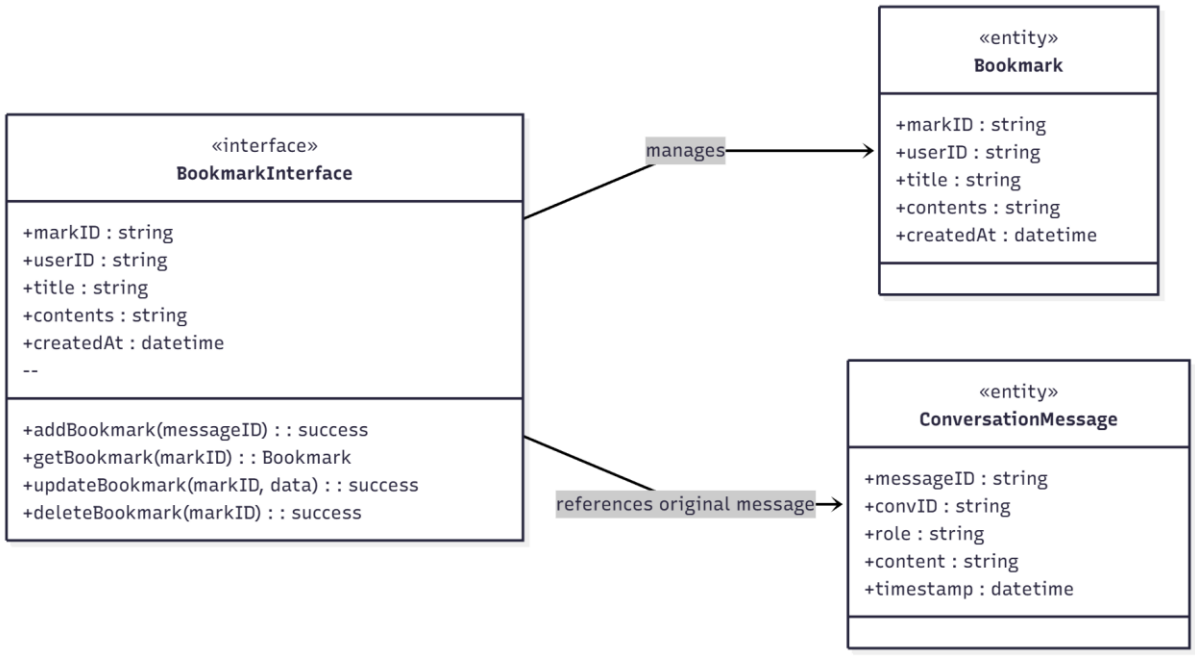

Figure 4.9: Class Diagram – Bookmark

Sequence Diagram

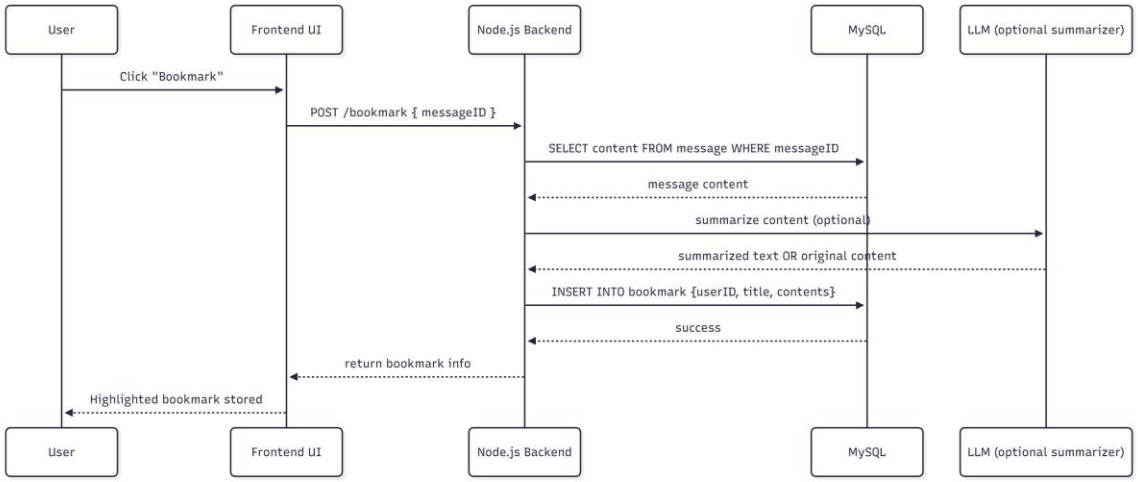

Figure 4.10: Sequence Diagram – Bookmark

4.1.6. Schedule Component

Schedule 컴포넌트는 사용자가 직접 등록하거나 북마크된 콘텐츠로부터 자동 생성된 일정을 관리한다. 일정은 학사 일정, 제출 기한, 약속, 시험 날짜 등 다양한 항목을 포함할 수 있으며, 추후 알림 또는 외부 캘린더 연동 기능 기반 구조로 설계되어 있다.

Objectives

- calendarID (Primary Key): 일정 식별자
- userID: 일정 소유 사용자
- title: 일정 제목 (북마크 기반 자동 생성 또는 사용자 지정 가능)
- description: 일정의 상세 설명
- startDate: 일정 시작 시간
- endDate: 일정 종료 시간
- type: 일정 카테고리 (예: "exam", "deadline", "reminder", "class")
- createdAt: 일정 생성 시각

Methods

- createSchedule()
- updateSchedule()
- deleteSchedule()
- getSchedule()
- listSchedules()

Class Diagram

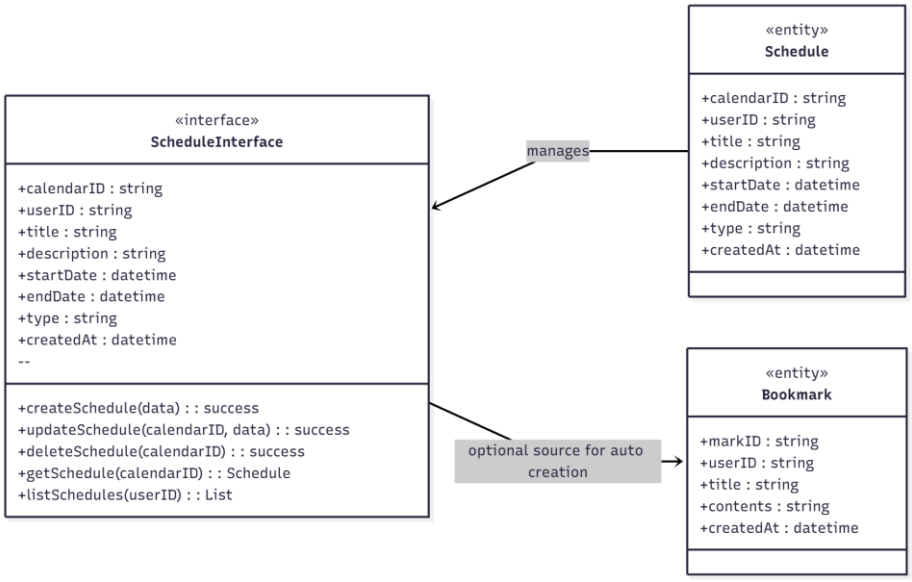

Figure 4.11: Class Diagram - Schedule

Sequence Diagram

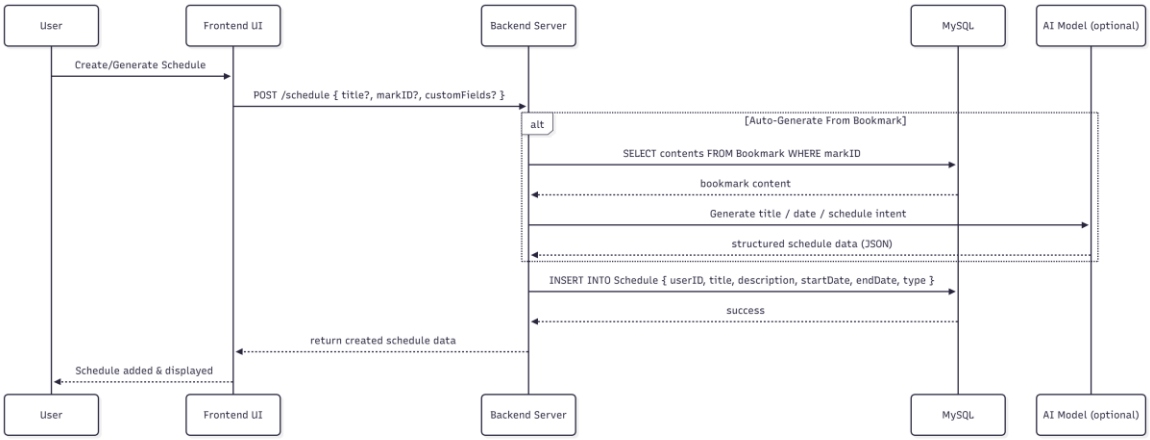

Figure 4.12: Sequence Diagram - Schedule

5. System Architecture – Backend

5.1. Objectives

이 챕터는 ASKku 서비스의 백엔드 시스템 구조를 기술한다. 백엔드는 사용자 인증, 질의 처리, 데이터 저장 및 검색, 벡터 기반 정보 검색, 대화 히스토리 저장, 사용자 시간표 및 공지 정보 제공 등의 기능을 처리한다. 모든 프론트엔드 요청은 백엔드 API 서버를 통해 처리되며, 서버는 외부 LLM 모듈 및 내부 데이터베이스 시스템과 연동된다.

5.2. Overall Structure

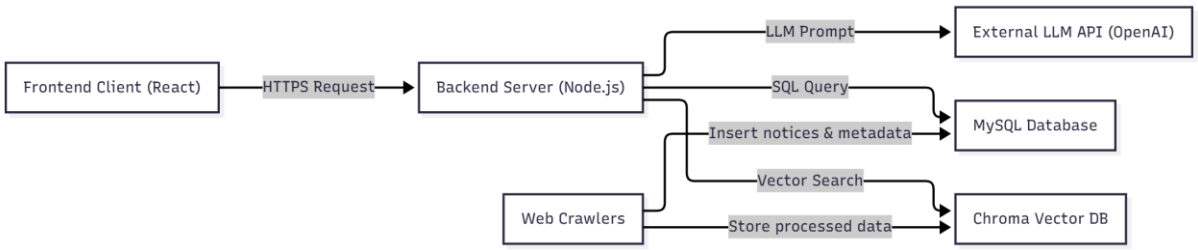

Figure 5.1: Overall Structure

ASKku 시스템의 백엔드 구조는 위와 같다. 사용자가 요청을 전달하면 서버(Node.js)는 해당 요청을 처리하기 위한 적절한 기능을 실행한다. 서버는 로그인 및 인증, 사용자 정보 관리, 대화 저장 및 검색, 공지 조회 요청을 처리하며, MySQL 데이터베이스를 이용하여 사용자 및 시스템 데이터를 지속적으로 저장한다. 또한 RAG 기반 검색 기능을 수행하기 위해 LangChain과 Chroma Vector Database를 사용하며, 최종 응답 생성을 위해 OpenAI LLM API와 연동된다.

5.3. Subcomponents

5.3.1. User Management System

User Management System은 회원가입, 로그인, 인증 상태 유지 및 사용자 정보를 관리하는 기능을 포함한다. 사용자가 처음 가입하면 입력된 정보는 MySQL DB에 저장되며, 비밀번호는 해싱(BCrypt) 처리된 형태로만 저장된다. 로그인 시 백엔드는 입력된 자격 증명을 검증하고, 성공적으로 인증될 경우 JWT 기반 Access Token을 발급하여 API 요청 시 인증에 활용한다.

Class diagram

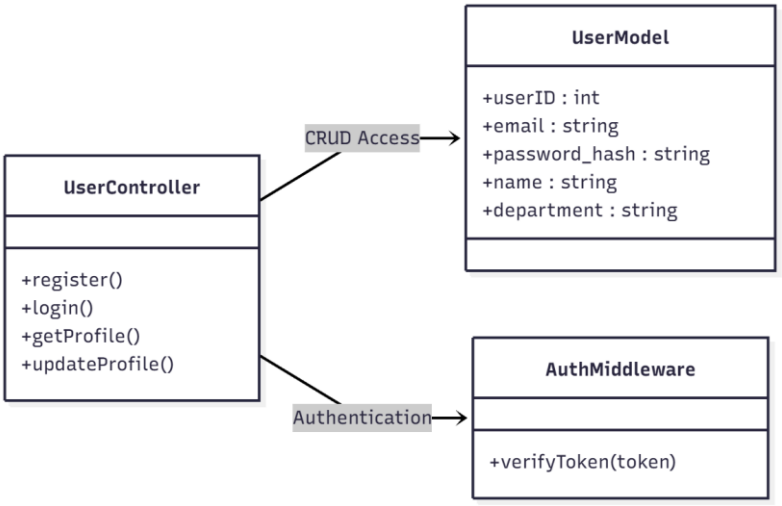

Figure 5.2: Class Diagram – User Management

Sequence Diagram

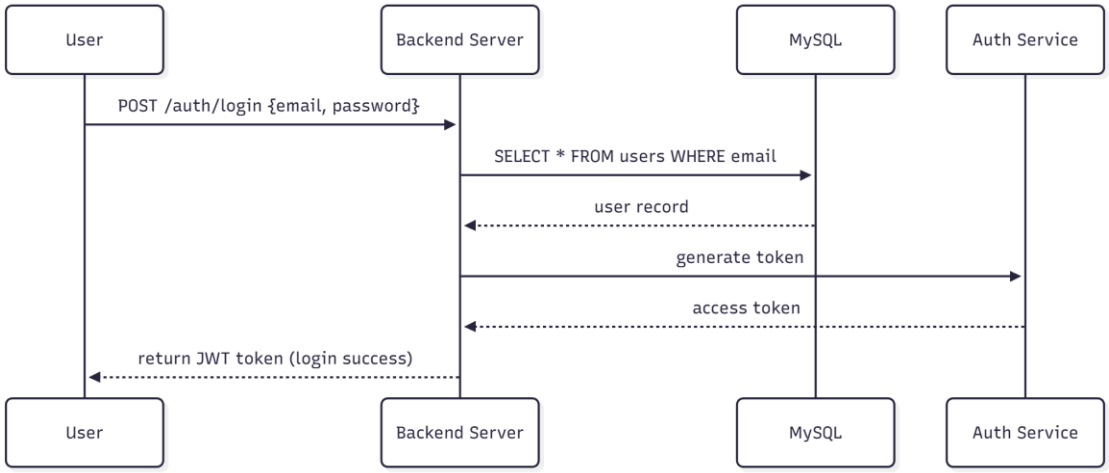

Figure 5.3: Sequence Diagram – User Management

5.3.2. Query Processing System (Conversation & Message System)

Query Processing System 은 사용자의 질문을 처리하고 AI 응답을 저장하는 핵심 모듈이다. 입력된 메시지는 Conversation 단위로 저장되며, 사용자 메시지와 AI 메시지가 각각 Message 테이블에 기록된다. 시스템은 사용자의 입력이 들어올 때 자동으로 Conversation 을 생성하거나 기존 세션에 응답을 이어 붙인다. 또한 이 컴포넌트는 Python 기반 RAG 서버와 통신하여 사용자 정보·시간표·이전 대화 기록을 포함한 컨텍스트 기반 AI 응답을 생성하도록 구성되어 있다.

Class Diagram

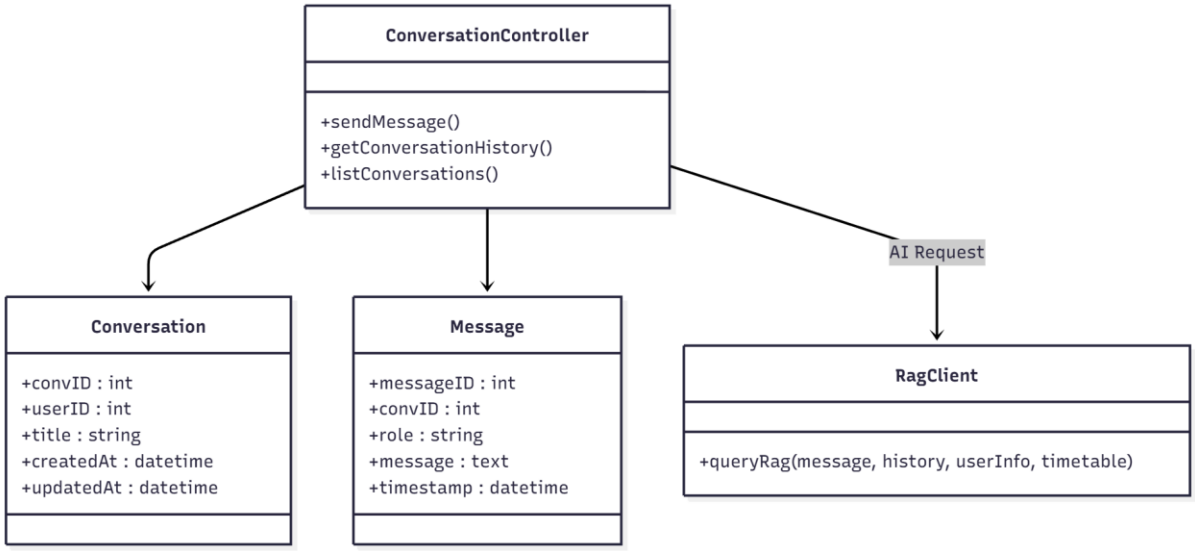

Figure 5.4: Class Diagram – Query Processing

Sequence Diagram

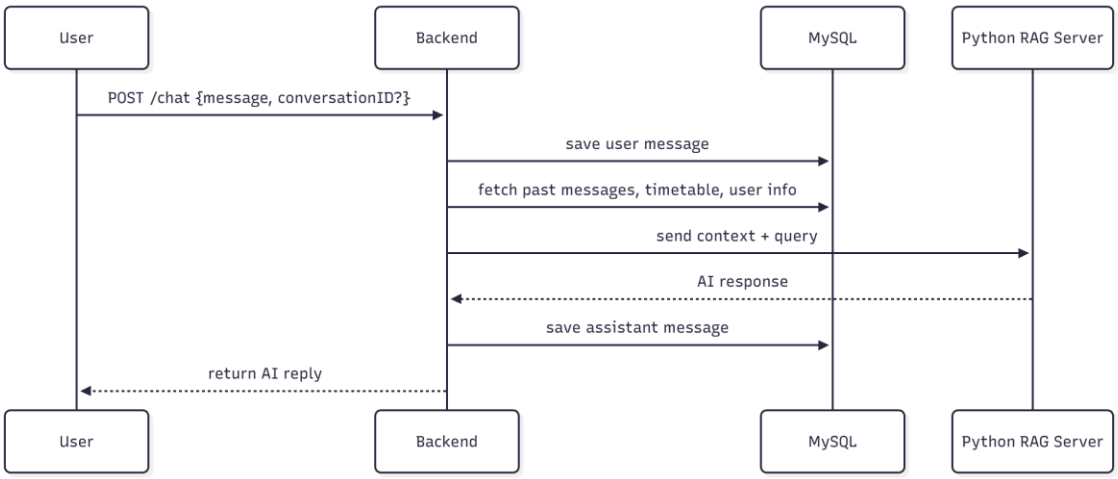

Figure 5.5: Sequence Diagram – Query Processing

5.3.3. Notice Retrieval System (RAG Data Layer)

Notice Retrieval System은 SKKU 학사 일정, 학과 공지사항, 크롤링된 PDF 문서 등 정보성 데이터를 RAG 시스템에 전달하고 검색하는 기능을 담당한다. 해당 시스템은 Node.js가 아닌 Python 기반 FastAPI 서버에서 수행되며, LangChain + ChromaDB 기반 벡터 검색과 OpenAI 모델을 사용한다.

Class Diagram

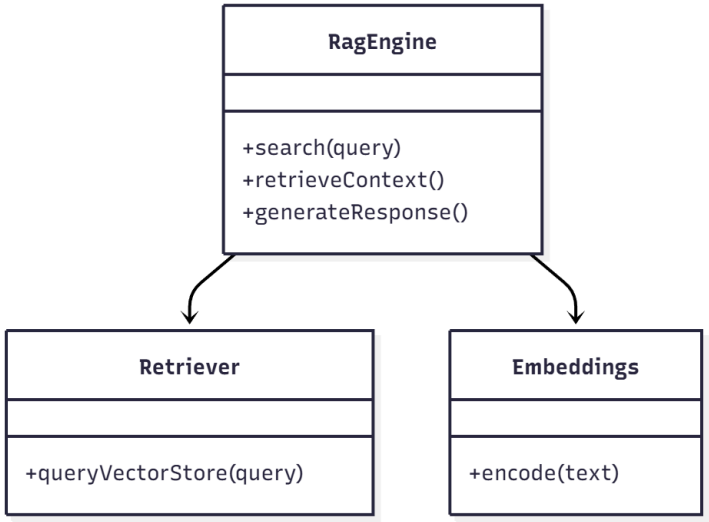

Figure 5.6: Class Diagram – Notice

Sequence Diagram

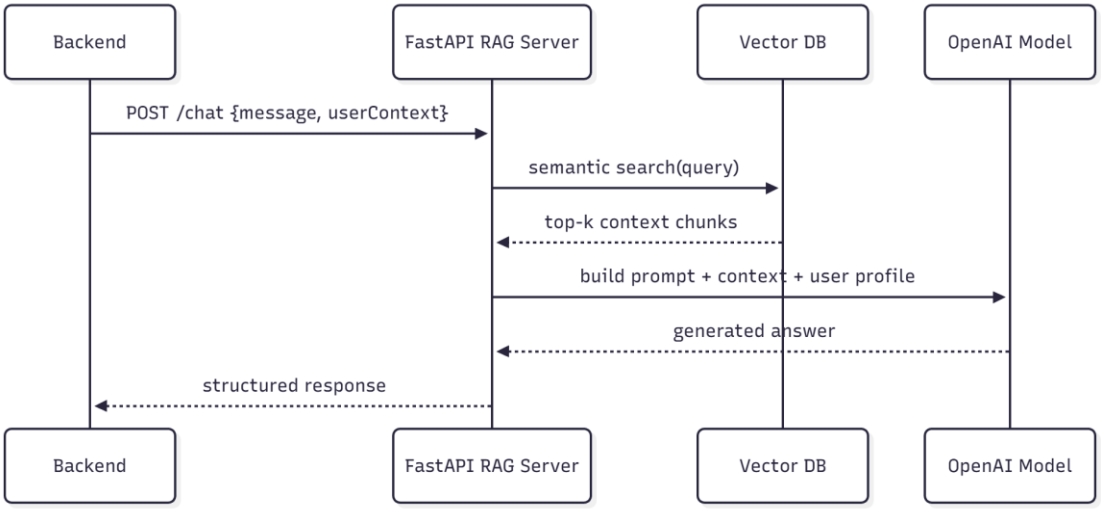

Figure 5.7: Sequence Diagram – Notice

5.3.4. Bookmark & Schedule System

Bookmark & Schedule System은 사용자가 대화 중 특정 메시지를 저장하고, 저장된 콘텐츠를 기반으로 일정을 생성하거나 관리할 수 있도록 설계된 통합 기능이다. 사용자는 원하는 메시지를 북마크하면, 시스템은 해당 내용을 저장하고 필요 시 LLM을 활용해 요약하거나 일정 후보를 생성할 수 있다. 북마크된 데이터는 검색과 분류에 활용되며, 일정 데이터는 캘린더 형태로 구조화되어 UI에 시각적으로 표시된다.

Class Diagram

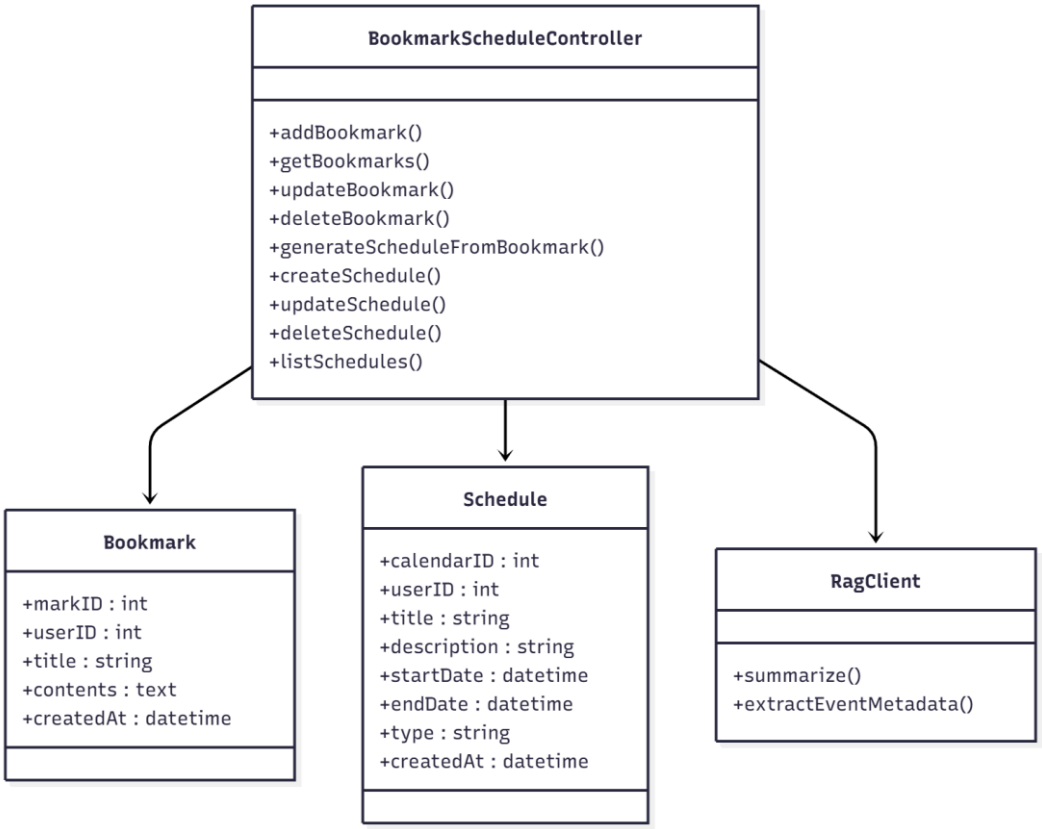

Figure 5.8: Class Diagram – Bookmark & Schedule

Sequence Diagram

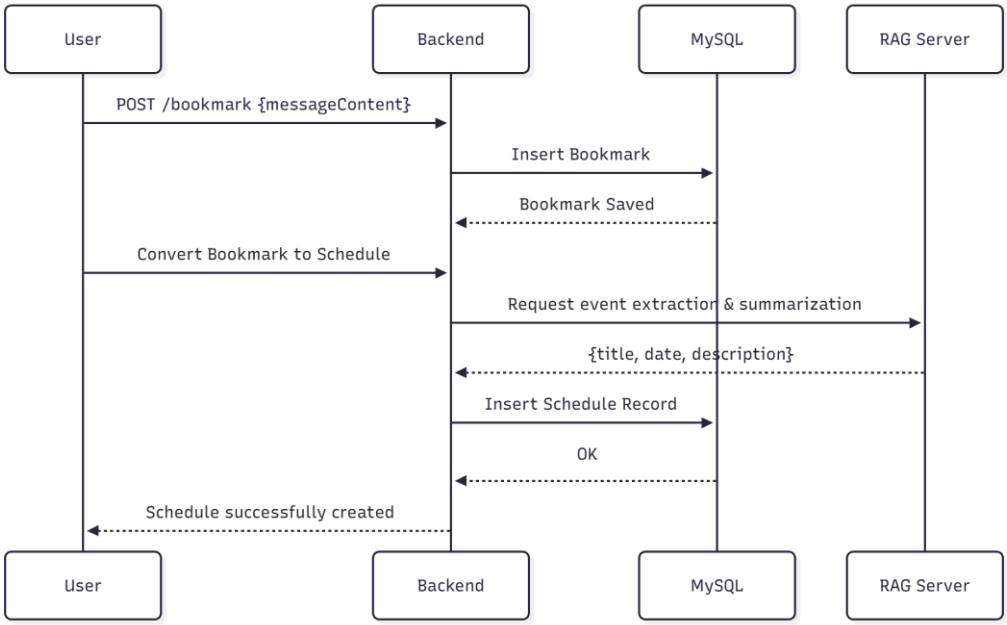

Figure 5.9: Sequence Diagram – Bookmark & Schedule

6. Protocol Design

6.1. Objectives

이 챕터는 프론트엔드 애플리케이션과 서버가 어떤 프로토콜로 상호작용하는지를 기술한다. 또한 서비스에서 사용되는 통신 인터페이스 정의 방식과 보안 구조를 포함하여, 요청과 응답이 어떤 형식으로 전달되는지를 명확히 한다.

6.2. AJAX

AJAX는 비동기 방식으로 서버와 데이터를 주고받는 기술이다. 이 방식은 전체 페이지를 새로고침 하지 않고 필요한 화면 요소만 갱신할 수 있도록 지원한다. JSON 형식 기반 데이터를 사용함으로써 사용자가 검색 질의를 입력하거나 사용자 정보를 수정하는 동안 인터페이스가 끊김 없이 동작할 수 있다.

6.3. Transport Layer

ASKku 시스템은 보안 통신을 위해 HTTPS를 사용한다. HTTPS는 HTTP 프로토콜에 TLS를 적용하여 전송되는 데이터를 암호화한다. 이를 통해 사용자 로그인 정보, 질의 내용, AI 응답 등이 전송되는 과정에서 데이터 위·변조 및 탈취를 방지한다.

Attribute	Detail
Protocol	HTTPS (TLS 1.2 이상)
Data Format	JSON (UTF-8)
Communication Type	REST API (Server ↔ Client), REST API (Node.js ↔ Python RAG Server)

Figure 6.1: Table of Transport Layer

6.4. Authentication

ASKku는 JWT 기반 인증 시스템을 사용한다. 사용자는 회원가입과 로그인을 통해 인증하며, 성공 시 서버는 Access Token을 반환한다. 이후 사용자는 API 요청 시 해당 토큰을 포함하여 인증을 유지한다.

6.4.1. Register

Request

Attribute	Detail	
Protocol	HTTPS (TLS 1.2 이상)	
Method	POST	
Request Body	student_no	Student ID

	email	User Email
	password	User Password
	name	User Name
	department	소프트웨어학과 고정

Figure 6.2: Table of Register Request

Response

Attribute	Detail
Success Code	201 CREATED
Failure Code	409 CONFLICT (중복 계정)
Success	{ "success": true, "message": "회원가입 완료" }
Failure	{ "success": false, "message": "이미 가입된 이메일입니다." }

Figure 6.3: Table of Register Response

6.4.2. Login

Request

Attribute	Detail	
Protocol	HTTPS (TLS 1.2 이상)	
Method	POST	
Request Body	Identifier	User Email
	password	User Password

Figure 6.4: Table of Login Request

Response

Attribute	Detail
Success Code	200 OK
Failure Code	401 Unauthorized
Success	{ "token": "JWT_TOKEN", "session": "active" }
Failure	{ "message": "Invalid credentials" }

Figure 6.5: Table of Login Response

6.4.3. Email Duplication Check

Request

Attribute	Detail	
Protocol	HTTPS (TLS 1.2 이상)	
Method	GET	
Request Body	Identifier	User Email

Figure 6.6: Table of Email Duplication Request

Response

Attribute	Detail
Success Code	200 OK
Success	{ "available": true/false }

Figure 6.7: Table of Email Duplication Response

6.5. Timetable API

6.5.1. Create Timetable

Request

Attribute	Detail	
Protocol	HTTPS (TLS 1.2 이상)	
Method	POST	
Request Body	season	시간표 학기 선택
	title (optional)	시간표 이름 선택 (선택사항)

Figure 6.8: Table of Timetable Create Request

Response

Attribute	Detail
Success Code	201 CREATED
Failure Code	500 Internal Server Error
Success	{ "success": true, "message": "시간표가 생성되었습니다.", "data": { ... } }
Failure	{ "success": false, "message": "서버 오류" }

Figure 6.9: Table of Timetable Create Response

6.5.2. Add Timetable Item

Request

Attribute	Detail	
Protocol	HTTPS (TLS 1.2 이상)	
Method	POST	
Request Body	courseName	강의 이름
	dayOfWeek	요일
	startTime	시작 시간
	endTime	종료 시간
	location	강의실 위치

Figure 6.10: Table of Add Timetable Item Request

Response

Attribute	Detail
Success Code	201 CREATED
Failure Code	403 Forbidden
Success	{ "success": true, "message": "과목이 추가되었습니다.", "data": {...} }

Figure 6.11: Table of Add Timetable Item Response

6.5.3. Delete Timetable Item

Request

Attribute	Detail	
Protocol	HTTPS (TLS 1.2 이상)	
Method	POST	
Request Body	itemID	시간표 항목 식별자

Figure 6.12: Table of Delete Timetable Item Request

Response

Attribute	Detail
Success Code	201 CREATED
Failure Code	403 Forbidden
	404 Not Found
Success	{ "success": true, "message": "과목 삭제 완료" }

Figure 6.13: Table of Delete Timetable Item Response

6.6. Notice & RAG Retrieval API

Request

Attribute	Detail
Protocol	HTTPS (TLS 1.2 이상)
Method	GET
Request Body	query

Figure 6.14: Table of Notice & RAG Retrieval Request

Response

Attribute	Detail
Success Code	200 OK
Failure Code	400 Bad Request
Success	{ "notices": [...], "count": ## }

Figure 6.15: Table of Notice & RAG Retrieval Response

6.7. Conversation + RAG API

Request

Attribute	Detail	
Protocol	HTTPS (TLS 1.2 이상)	
Method	POST	
Request Body	ConversationID	대화 식별자
	Message	대화 내용 (assistant)
	userInfo	사용자 정보 (시간표, 이전 내역)
	Timetable	시간표 정보
	Conversation_history	이전 대화 내역

Figure 6.16: Table of Conversation & RAG Request

Response

Attribute	Detail
Success Code	200 OK
Failure Code	502 (Python RAG 서버 연결 실패)
Success	{ "conversationID": ##, "reply": "AI Response Text" }

Figure 6.17: Table of Conversation & RAG Response

6.8. Bookmark Management API

6.8.1. Add Bookmark

Request

Attribute	Detail	
Protocol	HTTPS (TLS 1.2 이상)	
Method	POST	
Request Body	messageID	메시지 식별자
	title (optional)	북마크 이름

Figure 6.18: Table of Add Bookmark Request

Response

Attribute	Detail
Success Code	201 CREATED
Failure Code	400 Bad Request
Success	{ "success": true, "markID": <ID>, "message": "Bookmark saved" }

Figure 6.19: Table of Add Bookmark Response

6.9. Schedule Update System API

Request

Attribute	Detail	
Protocol	HTTPS (TLS 1.2 이상)	
Method	PATCH	
Request Body	title	일정 이름
	Description	일정 내용
	startDate	시작 날짜
	endDate	종료 날짜
	type	일정 종류 (공모전, 학내 활동 등)

Figure 6.20: Table of Schedule Update Request

Response

Attribute	Detail
Success Code	200 OK
Success	{ "message": "Schedule updated" }

Figure 6.21: Table of Schedule Update Response

7. Database Design

7.1. Objectives

7장에서는 시스템 데이터 구조와 이러한 구조가 데이터베이스로 어떻게 구현되는지에 대해 설명한다. 먼저 ER 다이어그램(Entity Relationship Diagram)을 통해 주요 엔티티와 그 관계를 식별한다. 그 다음 각 엔티티의 속성과 역할을 서술하고, 7.3절에서 관계형 스키마 및 SQL DDL을 기술한다.

7.2. ER Diagram

ASSKU의 데이터베이스는 다음과 같은 7개의 엔티티로 구성된다.

- USER: 시스템을 사용하는 학생 계정 정보
- CONVERSATION, MESSAGE: 챗봇 대화 세션 및 메시지 이력
- SCHEDULE: 개인 일정(수업 외 일정 포함)을 관리하는 캘린더
- TIMETABLE, TIMETABLE_ITEM: 학기별 시간표와 각 수업 시간 정보
- BOOKMARK: 사용자가 저장한 질의응답 북마크 정보

USER 엔티티가 중심이 되어, 한 명의 사용자는 여러 개의 대화(CONVERSATION), 메시지(MESSAGE), 일정(SCHEDULE), 시간표(TIMETABLE), 북마크(BOOKMARK)를 가질 수 있다. TIMETABLE과 TIMETABLE_ITEM은 1:N 관계로 구성되며, 하나의 시간표에 여러 개의 수업 항목이 속한다.

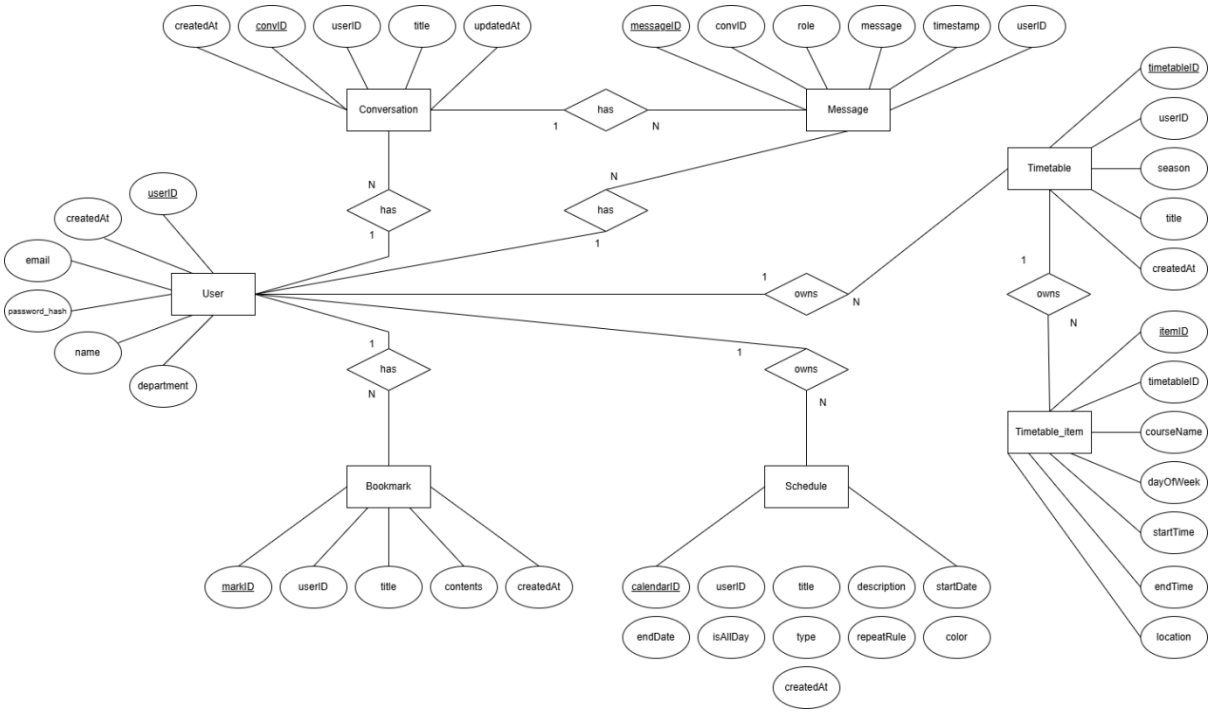

Figure 7.1: ER Diagram

7.2.1. User

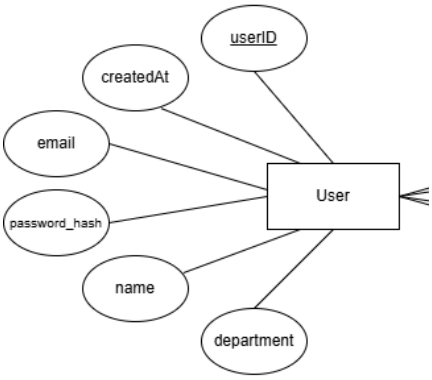

Figure 7.2: ER Diagram, Entity, User

User 엔티티는 ASKku 시스템을 사용하는 개별 사용자를 나타낸다. 로그인, 개인화된 질의응답, 시간표·일정 관리 등 대부분의 기능이 User를 기준으로 동작한다.

User는 아래의 엔티티들과 1:N 관계를 가진다.

- User (1) – (N) Conversation
- User (1) – (N) Bookmark
- User (1) – (N) Schedule
- User (1) – (N) Message
- User (1) – (N) Timetable

7.2.2. Conversation

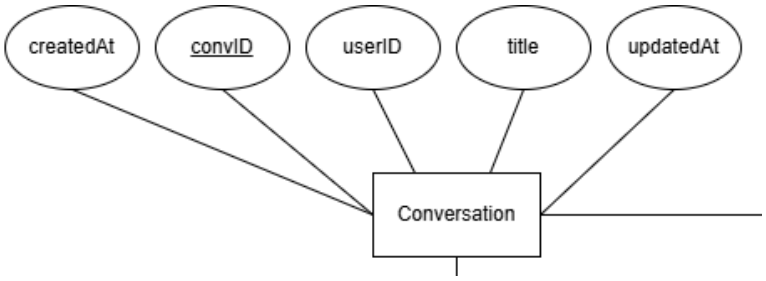

Figure 7.3: ER Diagram, Entity, Conversation

Conversation 엔티티는 사용자와 시스템 간의 하나의 대화 세션을 표현한다. 사용자가 ASKku에 접속하여 질문하고 답변을 받는 흐름을 하나의 Conversation으로 관리한다.

Conversation는 User와 N:1 관계를 가지며, Message와 1:N 관계를 가진다.

7.2.3. Message

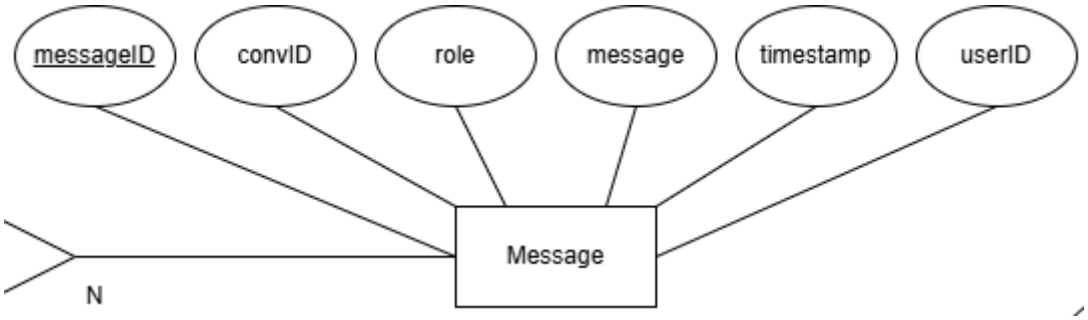

Figure 7.4: ER Diagram, Entity, Message

Message 엔티티는 하나의 Conversation을 구성하는 개별 메시지를 나타낸다. 사용자의 입력, 시스템(LLM)의 응답, 안내 메시지 등이 모두 Message로 저장된다. Message는 항상 하나의 Conversation에 속하며, 대화의 시간 순서를 created_at으로 관리한다.

7.2.4. Bookmark

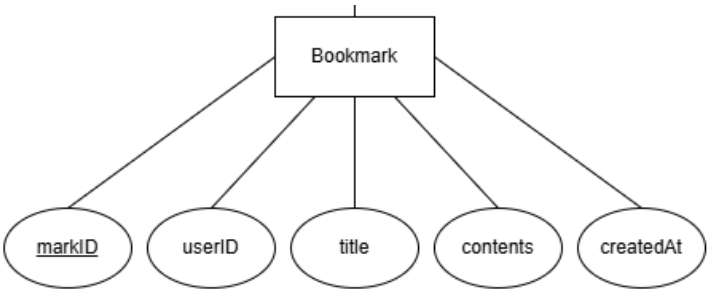

Figure 7.5: ER Diagram, Entity, Bookmark

Bookmark 엔티티는 사용자가 중요하다고 판단한 질문과 답변을 나중에 다시 보기 위해 저장하는 정보이다. 주요 Q/A를 빠르게 확인할 수 있도록 한다. Bookmark는 User와 N:1 관계를 가진다. 북마크는 당시의 질문·답변 내용을 그대로 snapshot 형태로 저장하며, Message와 직접적인 외래 키 관계는 두지 않는다.

7.2.5. Schedule

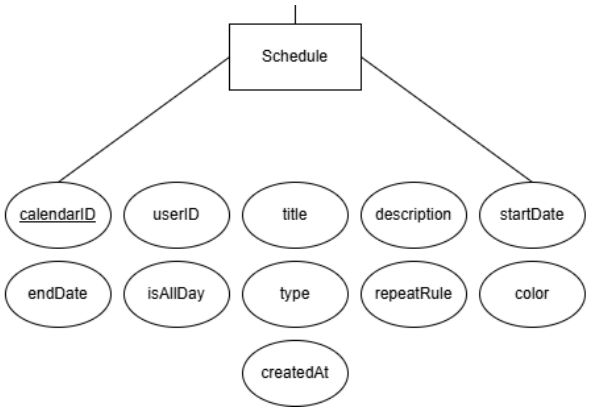

Figure 7.6: ER Diagram, Entity, Schedule

Schedule 엔티티는 사용자의 개인 일정을 저장하는 캘린더 항목을 나타낸다. 수업 외의 개인 일정, 행사 참가 일정, 알림 등이 여기에 포함된다. 하나의 User는 여러 개의 Schedule을 가질 수 있다.

다이어그램에서, 가진 Attribute가 많아 가장자리의 두 Attribute에만 선을 연결하여 표현하였다. 하지만 그 사이의 모든 Attributes가 Schedule에 속한다.

7.2.6. Timetable

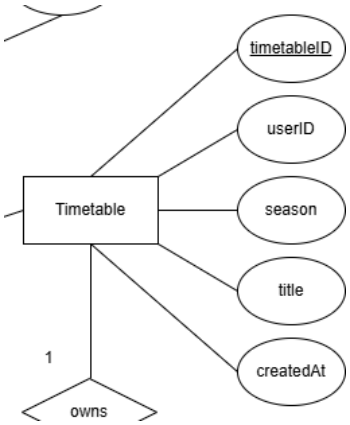

Figure 7.7: ER Diagram, Entity, Timetable

Timetable 엔티티는 특정 학기(시즌)에 대한 사용자 시간표를 나타낸다. 한 사용자가 여러 학기의 시간표를 가질 수 있으며, 각 시간표는 여러 Timetable_item으로 구성된다.

7.2.7. Timetable_item

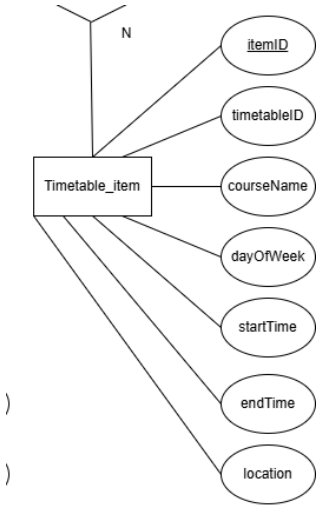

Figure 7.8: ER Diagram, Entity, Timetable_item

Timetable_item 엔티티는 시간표에 속하는 개별 수업(과목) 정보를 나타내며, Timetable_item은 Timetable과 N:1 관계를 가진다.

7.3. Relational Schema

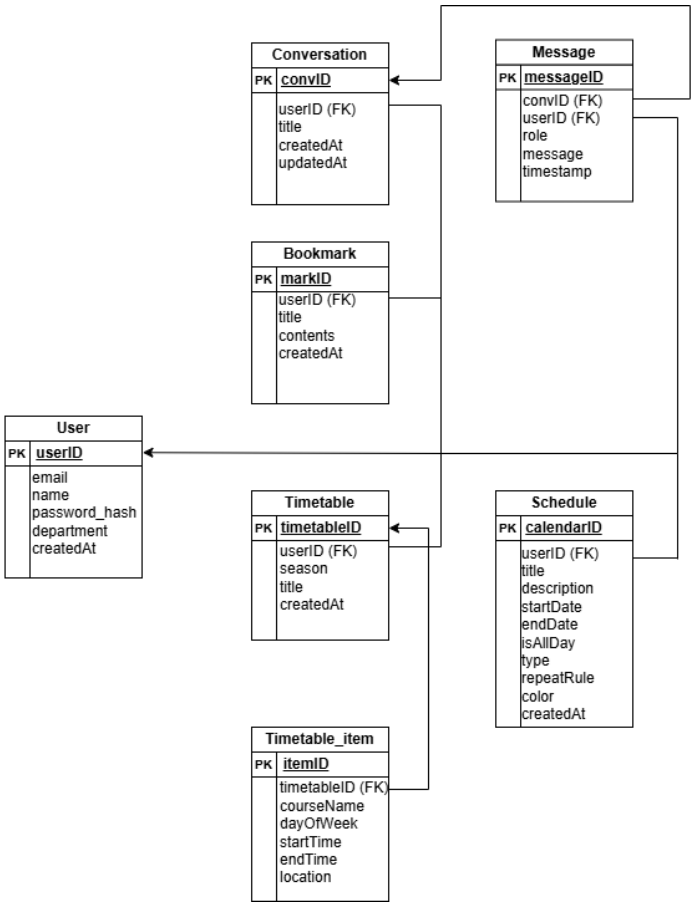

Figure 7.9: Relational Schema

7.4. SQL DDL

본 절에서는 7.3절에서 정의한 관계형 스키마를 실제 MySQL 상에서 구현하기 위한 SQL DDL(Data Definition Language)의 예시를 제시한다. 실제 구현에서는 각 테이블에 대한 CREATE TABLE 구문과 함께 인덱스, 외래 키 제약조건 등을 명시한다.

7.4.1. User

```sql
CREATE TABLE `USER` (
  `userID`        INT          NOT NULL AUTO_INCREMENT,
  `email`         VARCHAR(255) NOT NULL,
  `name`          VARCHAR(64)  NOT NULL,
  `password_hash` VARCHAR(255) NOT NULL,
  `department`    VARCHAR(64),
  `createdAt`     TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`userID`)
);
```

Figure 7.10: User SQL schema

7.4.2. Conversation

```sql
CREATE TABLE `CONVERSATION` (
  `convID`    INT          NOT NULL AUTO_INCREMENT,
  `userID`    INT          NOT NULL,
  `title`     VARCHAR(255),
  `createdAt` TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updatedAt` TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
                                    ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`convID`),
  CONSTRAINT `fk_conversation_user`
    FOREIGN KEY (`userID`) REFERENCES `USER` (`userID`)
);
```

Figure 7.11: Conversation SQL schema

7.4.3. Message

```sql
CREATE TABLE `MESSAGE` (
  `messageID` INT       NOT NULL AUTO_INCREMENT,
  `convID`    INT       NOT NULL,
  `userID`    INT,
  `role`      VARCHAR(32) NOT NULL,
  `message`   TEXT      NOT NULL,
  `timestamp` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`messageID`),
  CONSTRAINT `fk_message_conversation`
    FOREIGN KEY (`convID`) REFERENCES `CONVERSATION` (`convID`),
  CONSTRAINT `fk_message_user`
    FOREIGN KEY (`userID`) REFERENCES `USER` (`userID`)
);
```

Figure 7.12: Message SQL schema

7.4.4. Bookmark

```sql
CREATE TABLE `BOOKMARK` (
  `markID`    INT           NOT NULL AUTO_INCREMENT,
  `userID`    INT           NOT NULL,
  `title`     VARCHAR(255),
  `contents`  TEXT,
  `createdAt` TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`markID`),
  CONSTRAINT `fk_bookmark_user`
    FOREIGN KEY (`userID`) REFERENCES `USER` (`userID`)
);
```

Figure 7.13: Bookmark SQL schema

7.4.5. Schedule

```sql
CREATE TABLE `SCHEDULE` (
  `calendarID`  INT          NOT NULL AUTO_INCREMENT,
  `userID`      INT          NOT NULL,
  `title`       VARCHAR(255) NOT NULL,
  `description` TEXT,
  `startDate`   TIMESTAMP    NOT NULL,
  `endDate`     TIMESTAMP    NOT NULL,
  `isAllDay`    TINYINT(1)   NOT NULL,
  `type`        VARCHAR(64),
  `repeatRule`  VARCHAR(128),
  `color`       VARCHAR(32),
  `createdAt`   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`calendarID`),
  CONSTRAINT `fk_schedule_user`
    FOREIGN KEY (`userID`) REFERENCES `USER` (`userID`)
);
```

Figure 7.14: Schedule SQL schema

7.4.6. Timetable

```sql
CREATE TABLE `TIMETABLE` (
  `timetableID` INT          NOT NULL AUTO_INCREMENT,
  `userID`      INT          NOT NULL,
  `season`      VARCHAR(32),
  `title`       VARCHAR(255),
  `createdAt`   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`timetableID`),
  CONSTRAINT `fk_timetable_user`
    FOREIGN KEY (`userID`) REFERENCES `USER` (`userID`)
);
```

Figure 7.15: Timetable SQL schema

7.4.7. Timetable_item

```sql
CREATE TABLE `TIMETABLE_ITEM` (
  `itemID`      INT          NOT NULL AUTO_INCREMENT,
  `timetableID` INT          NOT NULL,
  `courseName`  VARCHAR(255),
  `dayOfWeek`   VARCHAR(16),
  `startTime`   TIME,
  `endTime`     TIME,
  `location`    VARCHAR(255),
  PRIMARY KEY (`itemID`),
  CONSTRAINT `fk_ttitem_timetable`
    FOREIGN KEY (`timetableID`) REFERENCES `TIMETABLE` (`timetableID`)
);
```

Figure 7.16: Timetable_item SQL schema

8. Testing Plan

8.1. Objectives

본 장에서는 ASKku 시스템의 품질을 확보하기 위한 테스트 계획을 기술한다. 테스트는 Development Testing, Release Testing, User Testing의 세 가지 주요 하위 그룹으로 구성되며, 이를 통해 다음을 목표로 한다.

- 기능적 요구사항 충족 여부 검증
- 성능(응답 시간) 및 신뢰성 검증
- 보안 및 개인정보 보호 측면 검증
- 실제 사용자 관점에서의 사용성(Usability) 검증

8.2. Testing Policy

ASKku의 테스트 정책은 개발 단계부터 배포 및 실제 사용 단계까지 순차적으로 진행되는 형태를 따른다.

- Development Testing
    - 모듈/컴포넌트 수준에서의 기능 및 품질 검증
- Release Testing
    - 통합 시스템의 안정성과 배포 가능 상태 검증
- User Testing
    - 실제 사용자들이 사용하는 환경에서의 사용성 및 만족도 검증

8.2.1. Development Testing

Development Testing은 시스템 개발 과정 전체에서 발생할 수 있는 오류를 조기에 발견하고 수정하기 위한 단계이다. 이 단계에서 소프트웨어는 지속적으로 변경·확장되므로, 프론트엔드, 백엔드, 데이터베이스, RAG 모듈 간의 인터페이스 오류가 발생할 가능성이 크다. 따라서 다음과 같은 관점에서 테스트를 수행한다.

- **Unit Test**
    - 백엔드 API(회원가입, 로그인, 질의응답, 일정 추천 등)에 대한 단위 테스트
    - Python 기반 RAG 모듈(크롤러, 임베딩, 검색 Logic)에 대한 함수 단위 테스트

- **Integration Test**
    - 프론트엔드 – 백엔드 – DB – RAG 모듈을 소규모로 연결한 상태에서의 상호 작용 검증
- **Code Review / Static Analysis**
    - 개발 팀원 간 코드 리뷰를 통해 버그, 보안 취약점, 성능 이슈를 조기에 발견
    - Lint 도구, 타입 체크 등을 활용하여 코드 품질 유지

Development Testing 단계에서는 특히 성능(Performance), 신뢰성(Reliability), 보안(Security) 측면의 기준을 정의하고, 이 기준을 만족하도록 반복적인 테스트와 개선을 수행한다.

8.2.2. Release Testing

Release Testing은 시스템이 실제로 배포될 수 있는 상태인지 확인하는 단계이다.

- 전체 기능을 통합한 상태에서의 시스템 테스트(System Test) 수행
- 새로운 기능 추가 또는 변경 이후, 기존 기능에 영향이 없는지 확인하기 위한 회귀 테스트(Regression Test) 수행

ASKku의 Release Testing에서는 다음과 같은 항목을 중심으로 테스트를 진행한다.

- 핵심 기능(질의응답, 북마크, 일정 관리, 공지 검색)의 정상 동작 여부
- 응답 시간 및 오류율 측정
- 장애 상황(외부 API 오류, DB 장애 등) 발생 시 예외 처리 동작 확인

8.2.3. User Testing

User Testing은 실제 사용자가 시스템을 사용하는 과정을 통해 실제 사용 시나리오에서의 문제점을 발견하고, 사용성을 개선하기 위한 단계이다.

- 대상
    - 내부 테스트 그룹
- 방법
    - 대표 시나리오(학사 정보 질문, 전공/교양 관련 질의, 행사 추천, 일정 등록 등)를 수행
    - 설문조사 및 인터뷰를 통한 피드백 수집
- 평가 항목
    - 응답의 이해도 및 신뢰성

    - 인터페이스의 직관성 및 편의성
    - 전체적인 만족도 및 개선 요구사항

8.2.4. Testing Case

테스트 케이스(Test Case)는 앞서 언급한 Performance, Reliability, Security, Usability를 중심으로 설계한다. 각 측면에 대해 최소 5개 이상의 테스트 케이스와 시나리오를 정의하고, 테스트 결과를 요약한 평가 보고서를 작성한다. 예를 들면 다음과 같다.

- Performance
    - Q/A 요청에 대한 평균 응답 시간 측정
    - 동시 접속자 수 증가에 따른 응답 시간 변화 측정
- Reliability
    - 장시간 연속 사용 시 오류 발생 여부 검증
    - 예외 상황(잘못된 입력, 네트워크 오류 등)에 대한 처리 검증
- Security
    - 인증되지 않은 사용자의 개인 정보 접근 시도 차단
    - 비밀번호 및 토큰 관리 검증

9. Development Plan

9.1. Objectives

본 장에서는 ASKku 시스템의 개발 환경 및 사용 기술, 제약사항과 가정에 대해 설명한다. 프론트엔드, 백엔드, 데이터베이스 및 AI 모듈에 사용되는 주요 기술 스택을 정리하고, 개발 시 고려해야 할 제약사항과 외부 의존성을 명시한다.

9.2. Frontend Environment

ASKku의 프론트엔드는 웹 브라우저 환경에서 동작하는 단일 페이지 어플리케이션(SPA)을 목표로 한다. 주요 타깃 브라우저는 최신 버전의 Chrome, Edge, Firefox, Safari이며, 데스크톱/노트북 환경을 기준으로 화면을 설계한다.

9.2.1. React

React는 컴포넌트 기반의 UI 라이브러리로, 상태 관리와 재사용 가능한 UI 구성을 지원한다. 본 소프트웨어에서는 사용자 질문 입력, 응답 표시, 북마크 목록, 일정 관리 화면 등을 각각의 React 컴포넌트로 구성하여 유지보수성과 확장성을 확보한다.

9.2.2. HyperText Markup Language (HTML)

HTML은 웹 페이지의 구조를 기술하는 마크업 언어이다. 제목, 단락, 목록 등 문서의 기본 구조를 정의하며, React 컴포넌트에서 렌더링되는 엘리먼트들의 기반이 된다. ASKku에서는 질의응답 영역, 공지 목록, 일정 달력 등의 레이아웃을 정의하는 데 사용된다.

9.2.3. Cascading Style Sheets (CSS)

CSS는 HTML 요소의 시각적인 표현을 정의하는 스타일 언어이다. 본 시스템에서는 Tailwind CSS 또는 유사한 CSS 유틸리티 프레임워크를 사용하여 레이아웃, 색상, 타이포그래피 등을 정의하고, 반응형 디자인을 통해 다양한 화면 크기에서의 사용성을 확보한다.

9.3. Backend Environment

ASKku의 백엔드는 RESTful API 서버와 RAG 파이프라인으로 구성된다.

9.3.1. Node.js API Server

Node.js 기반의 API 서버는 클라이언트 요청을 처리하고, MySQL 및 RAG 모듈과 연동하는 역할을 수행한다. 주요 역할로는 회원가입, 로그인, 사용자 정보 조회, 질의응답 요청 수신 및 Python RAG 모듈 호출, 북마크, 일정, 시간표 관련 CRUD API 제공이 있다.

9.3.2. Python RAG Module

Python 기반 RAG 모듈은 공지 및 학교 관련 문서를 수집·전처리하고, 임베딩 및 유사도 검색을 수행하는 역할을 담당한다.

- 주요 구성
    - 크롤러: 학교 공지/학사 정보 페이지에서 데이터 수집
    - 임베딩: 수집된 텍스트를 벡터로 변환
    - 검색: 사용자의 질문에 대해 관련 문서를 검색하여 LLM에 전달

9.3.3 Database (MySQL & ChromaDB)

- **MySQL**
    - 관계형 데이터 저장소로 User, Conversation, Message, Bookmark, Schedule, Timetable, Timetable_item 등 핵심 엔티티를 관리한다.
- **ChromaDB**
    - 벡터 스토어로, 공지 및 문서 임베딩을 저장하고 유사도 검색을 수행한다.

9.4 Constraints

ASKku 시스템은 다음과 같은 제약사항을 고려하여 설계·구현된다.

- 널리 사용되는 표준 기술(React, Node.js, Python, MySQL 등)을 우선적으로 사용한다.
- 사용자의 질의응답 요청에 대한 평균 응답 시간은 15초 이내를 목표로 한다.
- 로열티를 요구하거나 별도의 상용 라이선스가 필요한 기술은 가능한 사용을 지양한다.
- 시스템 비용과 유지보수 비용을 고려하여, 오픈소스 기반 솔루션을 우선 사용한다.
- 향후 기능 추가 및 데이터 증가를 고려하여 확장 가능한 구조로 개발한다.
- 개인정보 보호를 위해 비밀번호는 해시 처리하며, 모든 통신은 HTTPS를 사용한다.
- 코드 작성 시 유지보수성을 고려하여 네이밍 규칙 및 주석 작성 규칙을 준수한다.

9.5. Assumptions and Dependencies

본 문서에서 기술하는 시스템은 다음과 같은 가정과 외부 의존성을 기반으로 한다.

- 사용자는 최신 버전의 웹 브라우저와 안정적인 인터넷 환경을 사용한다고 가정한다.
- 시스템은 주로 데스크톱/노트북 환경에서 사용된다고 가정하며, 모바일 환경은 제한적인 수준에서만 지원될 수 있다.
- OpenAI API 및 ChromaDB 등 외부 서비스의 가용성을 전제로 하며, 해당 서비스 장애 시에는 기능 제한 또는 축약된 응답 제공이 발생할 수 있다.

10. Supporting Information

10.1. Software Design Specification

본 설계 명세서는 소프트웨어 요구사항 명세 표준 IEEE Std 830-1998과 소프트웨어 설계 명세 표준 IEEE Std 1016-2009에서 제시하는 권장 구조를 참조하여 문서 체계를 수립하였으며, 이전에 작성된 ASKku 소프트웨어 요구사항 명세서와의 일관성을 유지하도록 구성하였다. 설계 단계에서 사용된 RAG 아키텍처, 벡터 데이터베이스, LangChain 프레임워크, ChatGPT API 연동 등은 각 기술의 공식 문서와 관련 자료를 참고하여 반영하였다. 본 명세서는 ASKku 프로젝트의 개발팀, 시스템 관리 담당자, 수업 지도자 등 이해관계자가 공통적으로 참조할 수 있는 설계 기준 문서로 활용된다.

.

10.2. Document History

ASKku 설계 명세서의 작성 및 개정 이력을 아래 표와 같이 정리한다.

Date	Version	Description	Writer
2025-11-23	v1.0.0	1. Purpose - 2. Introduction	이동환
2025-11-24	v1.1.0	4. System Architecture - Frontend 5. System Architecture - Backend	김태완
2025-11-26	v1.2.0	3. System Architecture – Overall	이동환
2025-11-26	v1.3.0	6. Protocol Design	김태완
2025-11-27	v1.4.0	7. Database Design 개요	오은서
2025-11-27	v1.4.1	7. Database 다이어그램, schema 추가	김건
2025-11-29	v1.5.0	8. Testing Plan – 9. Development Plan	오은서
2025-11-30	v1.6.0	설계 명세서 취합 및 검토	팀 전원
2025-11-30	v1.6.1	Frontend, Backend, Protocol, Database 파트 보완	김태완, 김건